IMPLEMENTATION AND OPTIMIZATION OF DEEP LEARNING TECHNOLOGIES IN AI DRUG DISCOVERY / PHARMACEUTICAL FRAMEWORKS

DISSERTATION

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF

MASTER OF SCIENCE IN BIOINFORMATICS

By

ADNAN RAZA

(24MBI005)

UNDER THE SUPERVISION OF

Prof. K. Mustafa

Department of Computer Science

Jamia Millia Islamia, New Delhi

(Internal Supervisor)

AND

Dr. Gaurav Ahuja

Associate Professor

Indraprastha Institute of Information Technology Delhi (IIITD)

(External Supervisor)

DEPARTMENT OF COMPUTER SCIENCE

FACULTY OF SCIENCES

JAMIA MILLIA ISLAMIA

NEW DELHI

Year of Submission: 2026

DECLARATION

I, Adnan Raza, student of M.Sc. Bioinformatics, Jamia Millia Islamia, hereby declare that the report titled "Implementation and Optimization of Deep Learning Technologies in AI Drug Discovery / Pharmaceutical Frameworks", which is submitted by me to the Department of Computer Science, Faculty of Sciences, Jamia Millia Islamia, New Delhi, in partial fulfillment of the requirements for the award of the degree of Master of Science in Bioinformatics, is a record of original project work carried out by me.

This work has not previously formed the basis for the award of any Degree, Diploma, Fellowship, or other similar title or recognition. I further declare that the report contains no plagiarism and follows the prescribed similarity guidelines.

Place: New Delhi

Date: _____

Adnan Raza

Roll No.: 24MBI005

CERTIFICATE

On the basis of the declaration submitted by Mr. Adnan Raza, student of M.Sc. Bioinformatics, this is to certify that the project report entitled "Implementation and Optimization of Deep Learning Technologies in AI Drug Discovery / Pharmaceutical Frameworks" submitted to the Department of Computer Science, Faculty of Sciences, Jamia Millia Islamia, New Delhi, in partial fulfillment of the requirements for the award of the degree of Master of Science in Bioinformatics, is a faithful record of original project work carried out by him under my supervision.

To the best of my knowledge and belief, the work has reached the requisite standard for submission and has not been submitted in part or full to any other university or institute for the award of any degree or diploma.

Place: New Delhi

Date: _____

Prof. K. Mustafa

Internal Supervisor

Department of Computer Science

Jamia Millia Islamia, New Delhi

INDRAPRASTHA INSTITUTE *of*
INFORMATION TECHNOLOGY **DELHI**

Gaurav Ahuja, Ph.D

Associate Professor
Dept. of Computational Biology
Indraprastha Institute of Information
Technology-Delhi (IIIT-Delhi)
(A State University Established
by the Govt. of NCT of Delhi)
Okhla, Phase III, New Delhi
Pin Code: 110020, INDIA

Merck Scientific Excellence
Humboldt Fellow Germany
Global 3Rs Recipient USA
INSA Young Associate
EMBO Young Investigator
INSA Young Medal India
Ramalingaswami Fellow

📧: gaurav.ahuja@iiitd.ac.in
🌐: www.ahuja-lab.in/
📲: (+91)11-26907475

17 May 2026

Re: Letter of Thesis Completion

To whom it may concern

This is to certify that Mr. Adnan Raza has successfully completed his thesis work under my supervision at the Department of Computational Biology, Indraprastha Institute of Information Technology Delhi (IIIT-Delhi).

The title of his research is: **"Implementation and Optimization of Deep Learning Technologies in AI Drug Discovery / Pharmaceutical Frameworks."**

The work was carried out from 1 January 2026 to 15 May 2026. During this period, Mr. Raza demonstrated strong technical skills in deep learning and a keen understanding of its applications in drug discovery. He successfully implemented and optimized deep learning pipelines for pharmaceutical frameworks, contributing to ongoing research in the lab.

I wish him the best in his future endeavors.

Sincerely,

ACKNOWLEDGEMENT

In the name of Allah, the Most Gracious, the Most Merciful.

I convey my heartfelt thanks to my internal supervisor, Prof. K. Mustafa, Department of Computer Science, Jamia Millia Islamia, New Delhi. His visionary ideas, constructive criticism, and support were key factors that played an important role in deciding the course for my dissertation. His profound knowledge of complex architecture systems not only gave me clarity on many issues, but also motivated me to keep improving my standard of work.

I am immensely grateful to my external supervisor, Dr. Gaurav Ahuja, Associate Professor, IIIT Delhi. My project would not have reached completion without his professional knowledge, constant motivation, and personal guidance. It is a privilege working under such a great visionary who believes in making his students independent thinkers. He not only helped me improve my analytical abilities, but also taught me how to conduct rigorous scientific research.

A very special thank you from my heart must be extended to the PhD students from the Ahuja Lab for their patience, assistance, and perseverance with me through my days in the lab. Their spirit of fraternity was highly appreciable, especially during the intensive period of benchmarking and handling of high-pressure HPC cluster loads.

Prof. Mansaf Alam, Course Co-ordinator, deserves all of my deepest gratitude for always guiding and supporting me throughout my master's journey. I would also like to extend my sincere appreciation to the Department of Computer Science, Jamia Millia Islamia for providing me with an enriching academic environment that helped conduct this research.

I have been extremely fortunate in the company of peers and friends who added much value to the dissertation. My deepest gratitude to my fellow researcher and co-author, Yasser, in the development of web interface forEvOlf project. Shubhadeep Dhuari, my collaborator from senior PhD scholars in NucXplore project.

My academic career would not have been the same had it not been for the immense backing of my friends. I am immensely grateful to my best friends, Sakshi and Ritika for being there at IIITD giving me that much-needed sense of comfort and camaraderie. Big kudos go out

to my wonderful host and friend Sabeel who never fails to accommodate me at his place in the night whenever I ended up burning midnights while working on projects. I owe heartfelt thanks to my childhood friend, Nischaya Nath, and Misbah and Tabish who has always shown me brotherly love and affection in abundance.

There are no words which can do justice to the immense gratitude I owe to my family with an especially special mention to my sister Adiba who has always supported me in times of both success and failures.

Finally, I bow my head in reverence and thank the almighty God who gave me the strength, resilience, and mental acumen required to complete this task successfully.

> *"Heaven would never seal all of one's paths! There is no absolutely desperate situation in this world, there are only people who despair. The answer to resolving problems will always be in our own hands; when we inspect our inner hearts, we will find that the miraculous heavenly power was in ourselves all along! To discover oneself, to recognize oneself, and to rely on oneself!"*

Adnan Raza

Roll No.: 24MBI005

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CCSM	Chromatin Condensation and Structural Measurement
CI	Continuous Integration
CLI	Command-Line Interface
CSV	Comma-Separated Values
CUDA	Compute Unified Device Architecture
DL	Deep Learning
FASTA	FAST-All sequence format
GLCM	Gray-Level Co-occurrence Matrix
GPU	Graphics Processing Unit
GPCR	G-protein-coupled Receptor
HEIP	Histopathology Image Processing framework used for segmentation
HOG	Histogram of Oriented Gradients
HPC	High-Performance Computing
LBP	Local Binary Pattern
MAT	MATLAB matrix file format
ML	Machine Learning
NGINX	Web server and reverse proxy used for routing
PyO3	Rust bindings for Python interoperability
QSAR	Quantitative Structure–Activity Relationship
RGCI	Segmentation workflow component used in the NucXplore pipeline
SMILES	Simplified Molecular Input Line Entry System
VRAM	Video Random Access Memory
WGPU	Cross-platform GPU abstraction library
WGSL	WebGPU Shading Language

WSI	Whole-Slide Image
XGBoost	Extreme Gradient Boosting

ABSTRACT

AI and deep learning are extensively applied to drug discovery in areas such as molecule screening, ligand-receptor binding prediction, toxicity evaluation, and image-based biological investigation. Nevertheless, many models designed for drug discovery purposes are hard to deploy due to their reliance on sensitive software configuration, hand-written code, non-conventional data formats, and poor deployment capabilities. This thesis seeks to solve this problem not by designing new neural network models but by enhancing existing ones.

This project has two case studies. In the first one, a deployment pipeline was designed to deploy the EvOlf evolutionary-guided deep learning framework for agonist prediction of mammalian GPCRome. The system contains a React/TypeScript based web application, Django-based backend, MySQL-powered dataset explorer, Docker-based inference services, NGINX-based routing, and a Nextflow pipeline for local and batch processing. Ligand SMILES and GPCR FASTA file are used as input, and agonist/non-agonist prediction is returned in CSV format.

In the second place, the NucXplore Segmented Nucleus Featurizer using Python has been re-written as a Rust wheel library with PyO3 bindings, WGPU integration, and WGSL shader compilation. In the final benchmarking experiment involving 249 image/MAT tile pairs, the crop-enabled Python workflow took 7217.83 seconds. The Rust CPU version took 1788.00 seconds with crop export and 410.97 seconds without crop export, resulting in speedup factors of 4.04× and 17.56×, respectively. For the Rust GPU/WGPU version without crop export, it took 471.62 seconds, showing speedup by 15.30× from Python but still slower than the optimized Rust CPU version without crop export. Feature validation involved 10,088,493 numeric comparisons of 248

The complete NucXplore Nextflow pipeline was further tested on a 314MB whole-slide image, resulting in 1209 tiles, 286 filtered out, and 102,103 nuclei analyzed, taking about an hour for crop/filtering, segmentation, feature extraction, and prediction. In summary, this dissertation demonstrates how engineering-based approaches to implementing AI tools can enhance their availability, replicability, packaging, and performance when applied to pharmaceutics and bioinformatics studies.

Keywords: Artificial Intelligence, Deep Learning, Drug Discovery, GPCR, EvOlf, NucXplore, Docker, Nextflow, Rust, PyO3, WGPU, WGSL, Bioinformatics

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

The drug discovery process is expensive and time-consuming, as it includes several steps such as target identification, screening, validation, optimization, pre-clinical, and clinical trials. The use of computational methods is increasingly popular for performing many of these activities through prioritizing compounds before their experimental testing. AI, ML, and DL can be applied in this context because they are capable of learning from molecular structures, protein sequences, assays, biological descriptions, and microscopic images.

But a useful machine learning model is not only judged by how well it predicts. There are many biomedical AI models developed as research code that requires a particular Python environment, manual preprocessing, hardware-specific assumptions, and non-standard I/O format. They might work well in their original lab setting, but it might be difficult for other researchers to deploy and use them due to the reasons stated above.

This dissertation tries to fill that gap in the software and systems layers. It does not propose any new deep learning architectures but concentrates on improving the accessibility, packaging, deployment, reproducibility, and efficiency of current AI-powered components. This dissertation is structured around two case studies: EvOlf, a framework for predicting GPCR-ligand agonists; and NucXplore, an approach for segmenting nuclei images and featurizing them.

1.2 AI DRUG DISCOVERY AND BIOMEDICAL IMAGE ANALYSIS

The application of AI tools to predict ligand-receptor interactions, molecular properties, virtual screening, toxicity prediction, drug repositioning, and image analysis in biology is now a reality. A task related to the above applications includes predicting interactions between GPCRs and ligands. The reason why GPCRs are important pharmacological targets is that they are involved in signal transduction, neurotransmission, sensation, immune regulation,

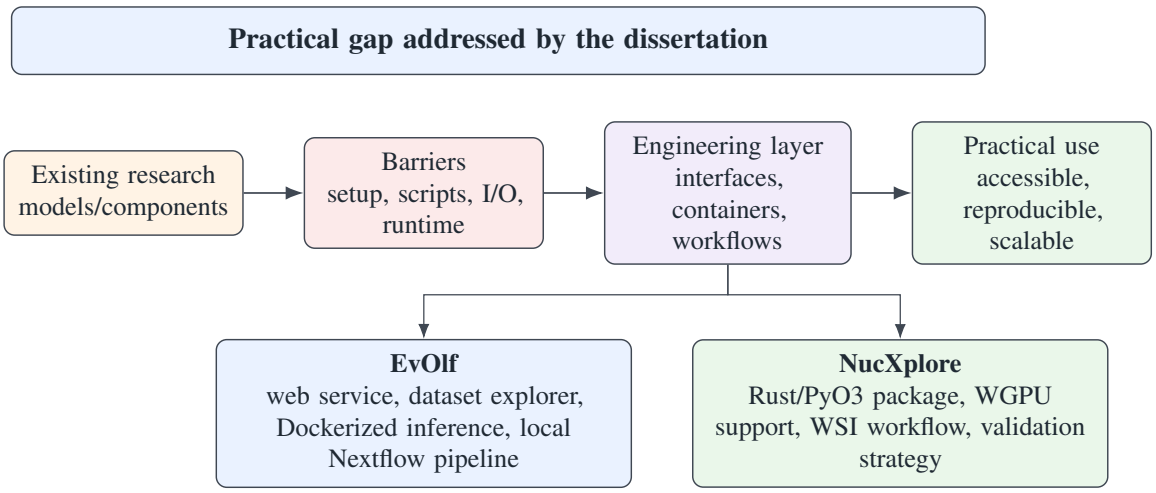

Figure 1.1: Problem-to-solution map showing how existing AI systems are connected to practical research use through deployment, workflow automation, and optimization.

and cardiovascular activity.

One other field is that of biomedical image analysis. Microscopy and histopathological images are used for understanding morphology, alterations due to disease, drug reactions, toxicity and phenotypic changes in cells. In these applications, nuclei segmentation is used to obtain descriptors such as shape, texture, staining, intensity, and spatial information. This information can be used for statistical analyses, classic machine learning algorithms, and even deep learning applications.

This is demonstrated by both areas, as there is a common requirement of needing a model/featurizer that is incorporated into a functional computational framework. Researchers require well-defined input, proper execution, controlled dependencies, understandable output, and replicable local or large scale workflow procedures. This leads to the adoption of web interfaces, Docker containers, workflow managers, validation processes, and performant implementation strategies.

1.3 SYSTEMS STUDIED IN THIS WORK

The first model to be presented here is EvOlf, which is an evolutionary-guided deep learning system that predicts agonists for mammalian GPCRome. In this dissertation, EvOlf was neither fine-tuned nor its architecture altered. Instead, a deployment component was added, including a web-based interface, dataset explorer, inference services containerized using Docker, and a reproducible Nextflow pipeline. This interface takes ligand SMILES and GPCR FASTA as input and gives out agonist/non-agonist prediction results.

The other system is NucXplore, an image featurization system for segmented microscopy and

histopathology images at the nucleus level. The initial featurizer had been implemented using Python. In order to optimize its performance and make a packaged version of the software, it has been rewritten using Rust language with PyO3 bindings that provide integration with Python. It now uses WGPU/WGSL for computation related to GPU.

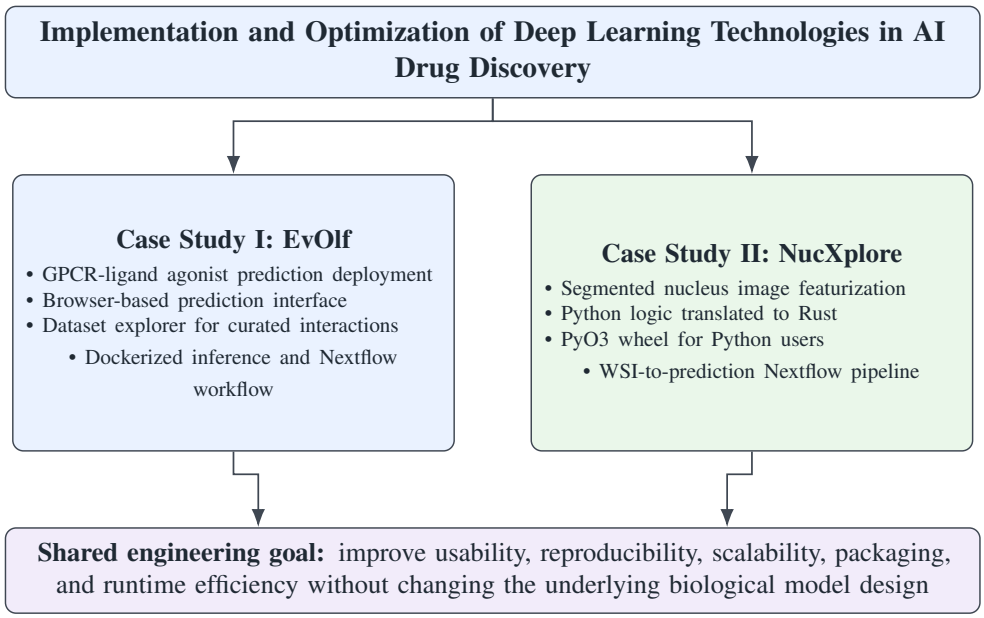

Figure 1.2: Overall thesis workflow showing the two implementation and optimization case studies.

1.4 CONTRIBUTION BOUNDARY

It is crucial to draw a distinction here because this is not an optimization thesis for the model. The purpose of this research does not involve improving the biological accuracy of EvOlf, creating a new architecture for predicting GPCRs, defining new biological features for NucXplore, training the segmentation model, or training the downstream XGBoost model. Rather, the contribution involves the engineering layer around these tools.

In the case of EvOlf, it is composed of prediction access for the users, dataset browsing, service segregation, containerization, batch processing locally, and export of the output. On the other hand, for NucXplore, it is composed of Rust/PyO3 library, batch processing, feature extraction via file input, optional cropping export, GPU-specific integration, validation contracts, and Nextflow orchestration.

1.5 PROBLEM STATEMENT

The key challenge being addressed in this dissertation is the lack of real-world usability in current AI and deep learning tools used by drug discovery and bioinformatics applications.

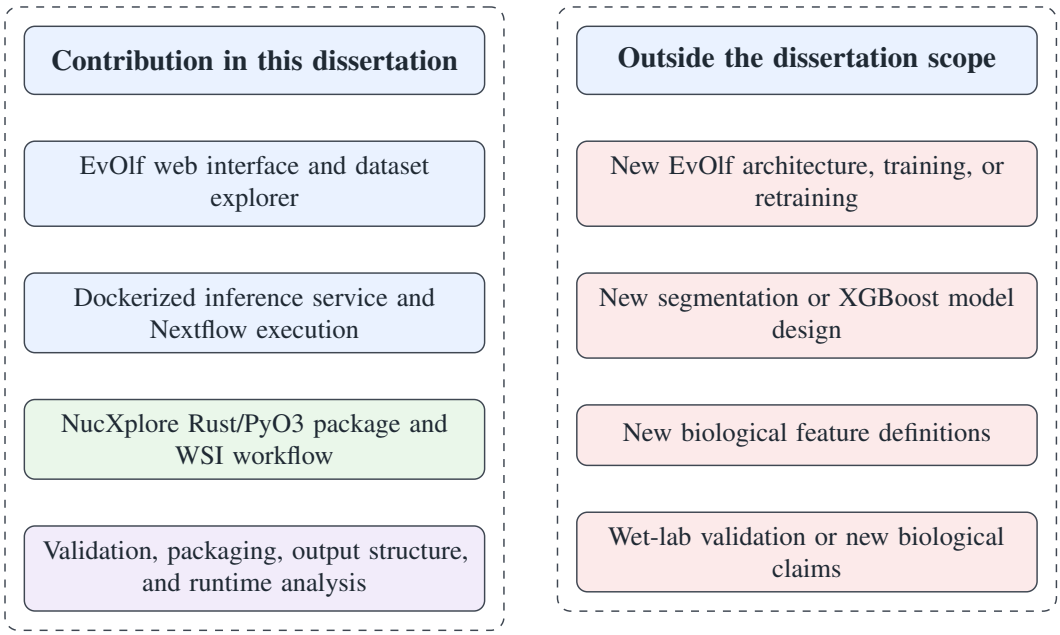

Figure 1.3: Boundary of the dissertation contribution. The work focuses on implementation, deployment, packaging, workflow execution, and optimization rather than new biological model development.

Existing research prototypes and featurizers typically rely on fragile execution environments, manual scripts, cumbersome data preprocessing steps, and highly non-standardized output formats.

EvOlf's problem statement involved making a multimodal GPCR-ligand predictor workflow available via a web interface and also ensuring that the workflow could be reproduced locally. On the other hand, NucXplore's problem statement focused on alleviating the drawbacks of a Python-based nucleus feature extraction tool without losing its compatibility with Python workflows. Consequently, the dissertation addresses deployment, workflow automation, containerization, packaging, and performance issues.

1.6 OBJECTIVES OF THE STUDY

The objectives of this study are:

1. To study implementation and optimization as practical requirements for AI drug discovery systems.
2. To develop a web interface and dataset explorer for EvOlf.
3. To containerize the EvOlf inference workflow and provide downloadable prediction outputs.
4. To develop a Nextflow pipeline for reproducible local and batch EvOlf execution.

5. To convert the NucXplore Python featurizer into a Rust + PyO3 wheel package.

6. To integrate WGPU/WGSL support and evaluate real-data feature-extraction performance across crop-enabled, no-crop, and GPU execution modes.

7. To validate the Rust implementation against reference feature outputs and connect NucXplore to a reproducible Nextflow workflow for WSI processing and prediction.

1.7 SCOPE OF THE WORK

The scope is restricted to the implementation, deployment, and optimization of the already existing systems. The project does not make any claims that it develops the original EvOlf architecture, trains the GPCR-ligand model, develops the biological features for NucXplore, or creates the segmentation and XGBoost prediction models that are used in NucXplore.
For EvOlf, the scope encompasses web interface, dataset explorer, prediction API, Docker implementation, and Nextflow pipeline. For NucXplore, the scope encompasses reimplementation using Rust/PyO3, package distribution, integration with GPU, integration with pipelines, reference output validation, and performance evaluation on real data. The primary contributions include usability, reproducibility, portability, and efficiency concerning existing AI-based drug discovery and biomedical image analysis tools.

CHAPTER 2

LITERATURE REVIEW

Drug discovery via AI tends to focus on model architectures, benchmarking results, and biological validation. All these are critical aspects, yet the practical implementation of such approaches relies on their deployability, replicability, scalability, and efficiency. In particular, it is even more important in the case of methods that integrate molecular strings, chemical graphs, protein sequences, assay information, or microscopy images into one pipeline. For this reason, drug discovery using AI should also be considered from the perspective of a system problem, involving data management, computing environment, workflow orchestration, and efficiency.[1–3]

2.1 AI DRUG DISCOVERY AS A SYSTEMS PROBLEM

Contemporary pipelines for AI-driven drug discovery are increasingly multimodal and infrastructure-based. The example of EvOlf is pertinent here since it was designed using a massive database of experimentally validated ligand-receptor interactions in mammalian organisms and was launched with platform and pipeline support for GPCR deorphanization.[3] Likewise, image-based drug discovery makes use of computational pipelines to convert cell and tissue images into numeric profiles for the purpose of studying mechanisms of action, toxicity testing, and drug response.[4,5]

The above examples clearly indicate that the surrounding software stack itself is part of the scientific utility of a model. Whether data validation, feature generation, inference, packaging, deployment, and reproducibility will enable a method to be useful outside of its laboratory setting. A model that relies on undocumented preprocessing or fragile dependencies is hard to reuse regardless of how well it performs on benchmarks.

This dissertation is accordingly situated at the level of implementation and optimization rather than novel model development. In addition to AI drug discovery, the relevant literature includes reproducible research, workflow systems, containerization, scientific interfaces, and

computational performance.

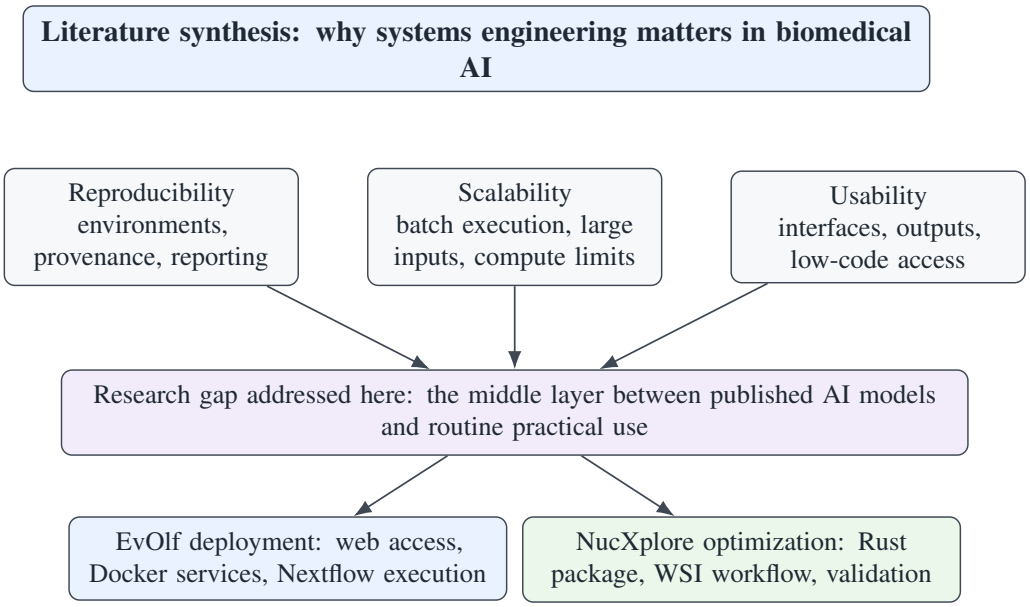

Figure 2.1: Synthesis of the reviewed literature into the core research gap: usable and reproducible biomedical AI requires infrastructure around the model itself.

2.2 REPRODUCIBILITY IN BIOMEDICAL AI

Reproducibility is an essential characteristic of computational sciences. Wilson et al. claim that scientific software must receive as much attention as experimental equipment since unstable software undermines scientific reliability.[6] Wilkinson et al. captured similar ideas in their formalization of the FAIR principles, which include findability, accessibility, interoperability, and reusability of research objects.[7] Reproducibility of computational biology research is additionally constrained by software versioning, data provenance, execution environment, and workflow logic.[8]

Biomedical AI applications, in particular, are highly dependent on accurate reporting of preprocessing and execution environment specifics. Fell et al. illustrated that reproducing digital pathology algorithms may be complicated due to minor differences in methodology, in particular in whole-slide image processing pipelines.[9] Workflow management systems help overcome this challenge by making analysis logic explicit and transportable. Wratten et al. consider workflow managers as key components of reproducible, scalable, and sharable biomedical analyses.[1]

In terms of this paper, reproducibility is considered to be a multi-layered concept. The input formats should be defined, the execution environment should be controlled, the workflow layers should be defined, and the outputs should be observable. The multi-layered approach

inspires the application of Docker containers, Conda environments, Nextflow, and parameter definitions.

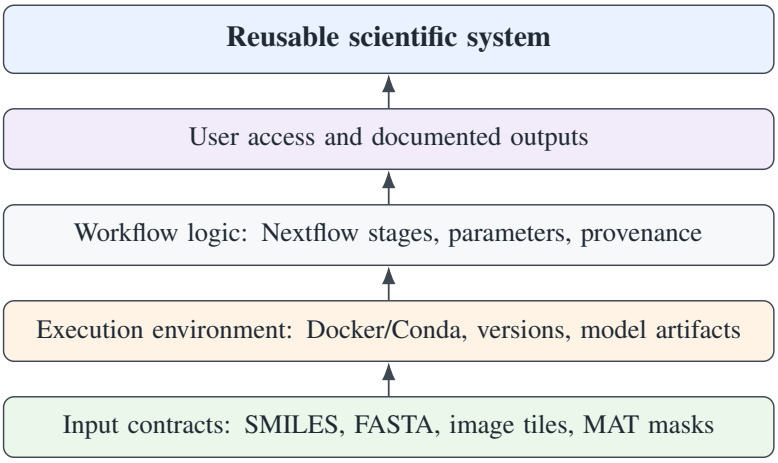

Figure 2.2: Reproducibility stack used as a conceptual guide in the dissertation. Reliable reuse requires control of inputs, environments, workflow logic, outputs, and user access.

2.3 SCALABILITY, COMPUTE SCARCITY, AND OPTIMIZATION

Scalability is what will allow the AI approach to transition from being used in a demo to routine usage. High-throughput imaging, multimodal molecule modeling, and large-scale screening pose compute, memory, storage, and I/O requirements. Efficient molecular AI literature is increasingly aware of the importance of memory efficiency, compute cost, and latency for deployment in resource-limited settings.[10,11] Compute shortage further exacerbates the mismatch between industry and academia.[12]

When it comes to multimodal drug discovery pipelines, the computational overhead is spread throughout the whole process and not just confined to the neural network part. Data validation, feature engineering, inference, packaging, re-running locally, and batch prediction all require resources. Therefore, optimization should consider lightweight setups, fewer dependencies, better scheduling, I/O control, and efficient implementation of compute-heavy sub-processes.

2.4 CONTAINERIZATION AND WORKFLOW MANAGEMENT

One approach to addressing environment-related issues in reproducibility is containerization. Boettiger refers to Docker as a technology that enhances portability, modularity, versioning, and reproducibility in computational science.[13] Similarly, Moreau et al. refer to containers as an efficient approach to dealing with complex and ever-changing software environments.[2] Applications packaged with their dependencies are not dependent on the operating environment.

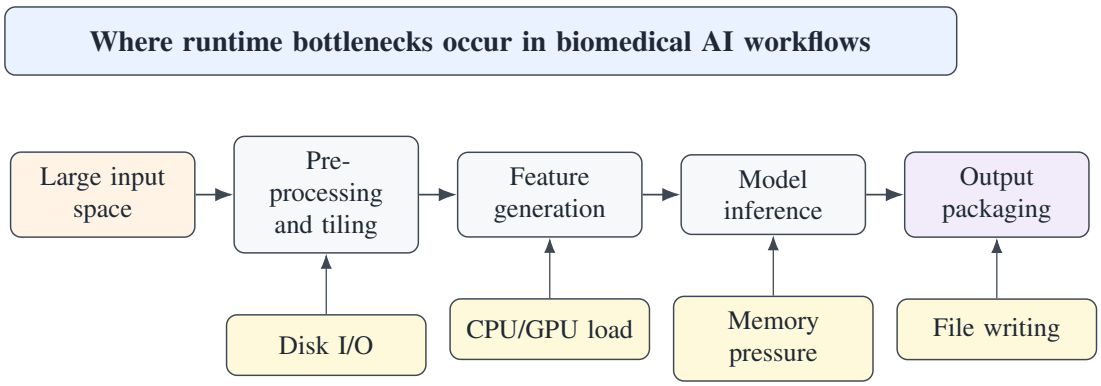

Figure 2.3: Typical bottleneck chain in multimodal biomedical AI workflows. Optimization must consider pre-processing, feature generation, inference, output writing, and data movement rather than only the neural network.

Workflow management systems address a different but related issue; they document the analysis process. Nextflow was proposed as a reproducible computational workflow management system that works both locally and on high-performance computing and cloud infrastructures.[14] In the context of AI drug discovery, input processing, feature extraction, model inference, and result generation would be documented as workflow steps.

Microservices architecture can also be considered in deployed scientific systems. The web front end, the API back end, the database, the model inference back end, and the file output layer could each have their own independent evolution. Decoupling them will make maintenance simpler.

2.5 ACCESSIBLE INTERFACES FOR SCIENTIFIC MODEL DEPLOYMENT

Accessibility of scientific software affects the users of a computational approach. This is exemplified by Galaxy, which is a well-known solution for web-based, reproducible, and transparent life sciences analysis.[15,16] Similarly, digital pathology solutions like QuPath and HistomicsTK demonstrate the importance of interactive exploration combined with the reusability of the underlying computational pipeline.[17,18]

Both these approaches are useful in the context of drug discovery. While some users may require single-prediction interactive exploration, others will require a reproducible batch execution mode. A web interface is convenient as it takes care of input validation, execution, output formatting, and downloading the results. The workflow mode can be used for larger-scale screening and local execution. The EvOlf part of this dissertation is structured along these lines.

2.6 GPCR–LIGAND PREDICTION AND DEPLOYMENT NEEDS

GPCRs constitute one of the most critical protein families in the drug development domain due to their wide-ranging regulatory effects and involvement in a majority of medications. GPCR–ligand predictions can help with receptor deorphanization, ligand discovery, and prioritizing targets for further study. EvOlf offers a current example of a multimodal approach to GPCR predictions based on experimentally confirmed interactions between receptors and ligands in mammals.[3]

The issue of evaluating different architectures of GPCR models is irrelevant to the scope of this work. What matters is that an intensive model can become practical. A practical GPCR prediction tool needs to accept SMILES representations of ligands, sequences of receptors, multiple feature extraction steps, inference, scoring, formatting results, and exporting them. Web access, Docker-based services, and Nextflow execution solve this deployment problem without modifying the model itself.

2.7 MICROSCOPY IMAGE FEATURIZATION

Biomedical imaging has evolved from manual analysis of small images to high throughput quantitative pipelines. High throughput microscopy can be used to produce large, multidimensional datasets, necessitating computational analysis.[19] Image-based phenotyping can transform cellular morphologies to feature vectors for drug discovery and biological analysis.[4] Nuclear features can be informative because nuclear morphology, texture, staining pattern, and spatial relationship can reveal the state of the cell or pathological changes in cells. Nuclear morphological, textural, staining, spatial, and chromatin features are measurable by CellProfiler, among other tools.[19,20] NucXplore can be contextualized within this framework because it measures nucleus-specific morphological, textural, staining, spatial, and chromatin features.

Feature calculation becomes a performance bottleneck at scale due to the repetitive nature of feature calculation on thousands of nuclei and tiled images. While Python is still useful for scientific workflows, dependency-heavy or loop-heavy approaches may hinder use due to installation or runtime issues. From a systems perspective, it makes sense to retain Python usability while moving heavy computation elsewhere.

2.8 SYNTHESIS AND RESEARCH GAP

There is clearly a need for the gap between model development and its application to be bridged. Many AI algorithms are assessed using only their architecture and performance on benchmarks, but little consideration is given to their deployment capabilities, packaging, workflow execution, downloading, performance bottlenecks, etc. These aspects make a difference when considering the reusability of a method.

This dissertation focuses on this intermediate stage. For the EvOlf case study, the contribution lies in no-code interface, data exploration, Dockerized inference, and Nextflow-based execution. For the NucXplore case study, the contribution consists in packaging, Rust/PyO3 implementation, workflow integration, validation, and runtime assessment. Altogether, these case studies place the dissertation right at the intersection of bioinformatics, AI drug discovery infrastructure, scientific software engineering, and computational optimization.

CHAPTER 3

METHODOLOGY

The methodology was built based on two case studies, which had similar aims: enhancing the applicability of the existing AI-based biomedical systems. In particular, EvOlf needed a deployment and reproducibility layer for a GPCR-ligand prediction model. On the other hand, NucXplore needed a packaging and performance-aimed reimplementation of a nucleus featurizer and then integration into the whole slide image pipeline. Thus, the work involved web development, containerization, workflow creation, Rust/PyO3 implementation, and GPU integration.

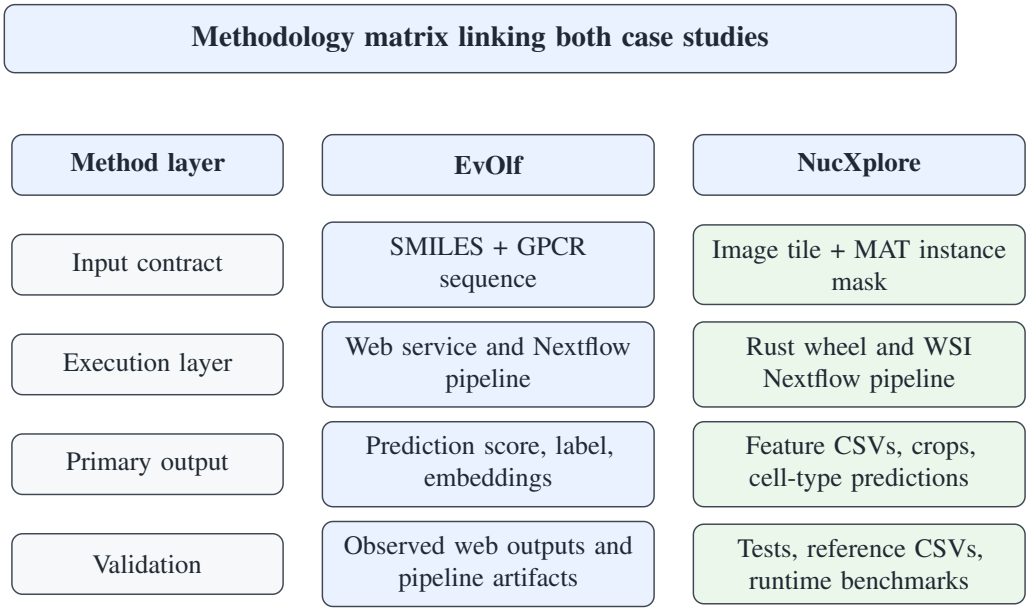

Figure 3.1: Methodology matrix showing how both case studies were evaluated using input contracts, execution layers, outputs, and validation evidence.

3.1 EVOLF WEB SERVICE AND NEXTFLOW PIPELINE METHODOLOGY

EvOlf was considered an already existing prediction tool and not something to redesign. The aim was to make the prediction process available via a web application and reproducible using a Nextflow pipeline. Two different methods were created to achieve the above-mentioned aim:

a web service for prediction via a web interface and a Nextflow pipeline.

3.1.1 WEB SERVICE DESIGN

The EvOlf web service was designed as a multi-component application. React and TypeScript were employed as the frontend technologies, while Django was used as the backend framework to implement the application logic. Structured data is stored in MySQL database, services are packed in Docker containers, and NGINX serves and routes the requests.

The web interface offers two types of functionality: the dataset explorer and the prediction service. The dataset explorer enables users to examine model-associated datasets without the need to open the files manually. Prediction service takes a ligand SMILES string and a GPCR FASTA sequence. The backend checks the provided input, triggers all necessary feature generation operations, does the inference on the trained EvOlf model, and sends back the output to the user. The output consists of an agonist/non-agonist label, a prediction score, and CSV files in a ZIP archive.

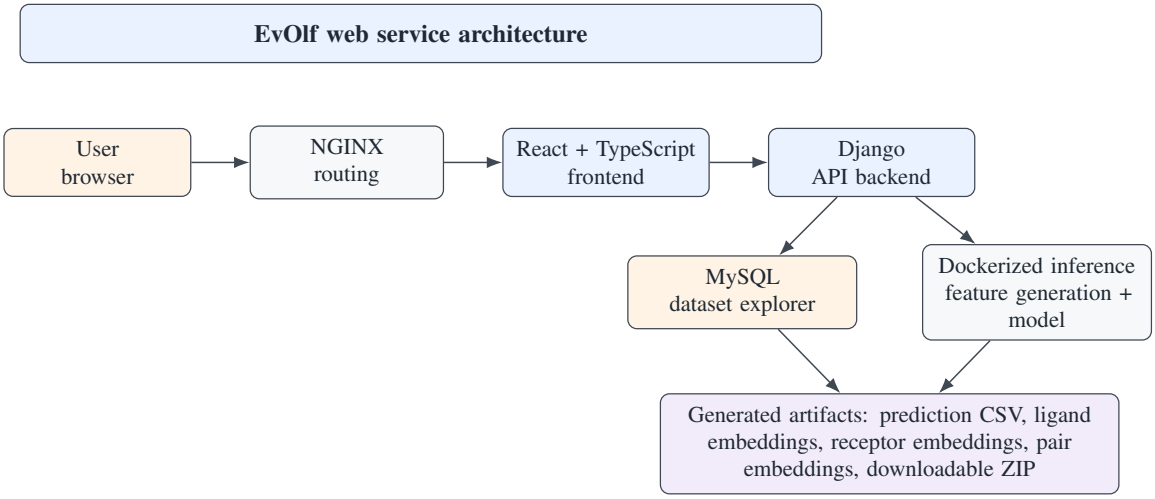

Figure 3.2: Web service architecture developed for EvOlf dataset exploration and online GPCR-ligand prediction.

3.1.2 EVOLF INPUT AND OUTPUT CONTRACT

Mapping of ligand records, receptor records, and ligand-receptor pairs is a key requirement of EvOlf. When using the web service, users submit a single ligand SMILES string and a receptor FASTA string. With the use of the pipeline mode, inputs will be supplied via a CSV file with ligand and receptor details. Identifiers could be defined by users or generated to maintain traceability.

Standardization of the inputs is needed to validate the required fields, generate the ligand and

receptor files, and ensure proper mapping of each pair. This process is needed since ligand and receptor featurizers operate separately prior to their joining for modeling. The outputs generated by EvOlf will allow for human interpretability and reusability. These outputs are the predictions themselves, which are viewable through the interface, and the feature files and predictions CSV files.

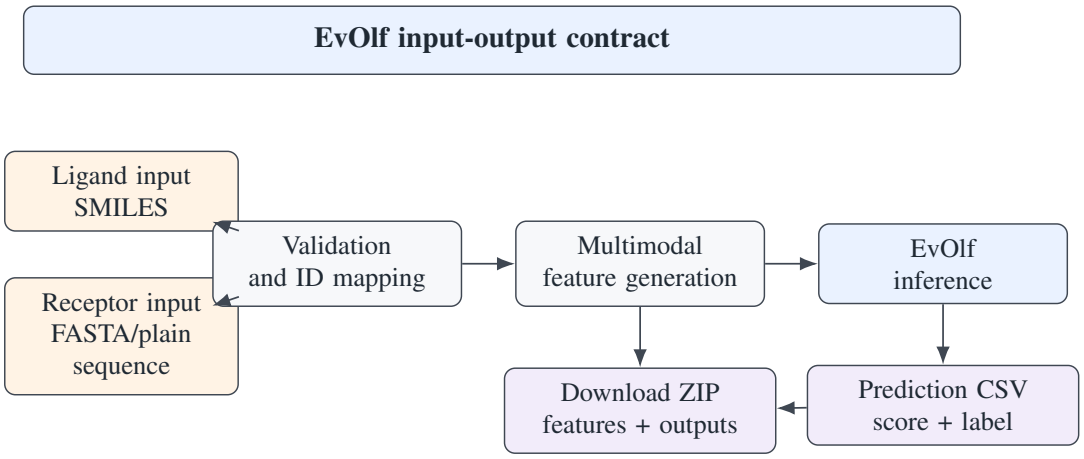

Figure 3.3: Input-output contract for the EvOlf deployment layer. The same logical contract is used by the web service and the local Nextflow workflow.

3.1.3 EVOLF FEATURE GENERATION AND INFERENCE

The EvOlf prediction pipeline involves several ligand and receptor feature generators. The list of ligand features includes ChemBERTa, Signaturizer, Mol2Vec, Mordred, and Graph2Vec. Receptor features consist of ProtBERT, ProtT5, ProtR, and MathFeaturizer. Each of these feature generators provides a different view on molecular and protein sequence data.

Once the feature generation is done, ligand and receptor outputs are merged and used as input to the feature structure expected by EvOlf. The generated features are further used as input to the pre-trained model, which produces the probability score and the binary label. The work on implementing EvOlf was aimed at providing a reproducible workflow rather than model modification.

3.1.4 DOCKER AND NEXTFLOW EXECUTION

To address dependency issues associated with various feature generation tools, Docker was utilized. Different stages of the project involve the use of different libraries and runtime environments; hence, using containers enhances portability and ease of maintenance. Service separation is also helpful when developing the web application, where responsibilities for front-end, back-end, database, routing, and inference can be separated.

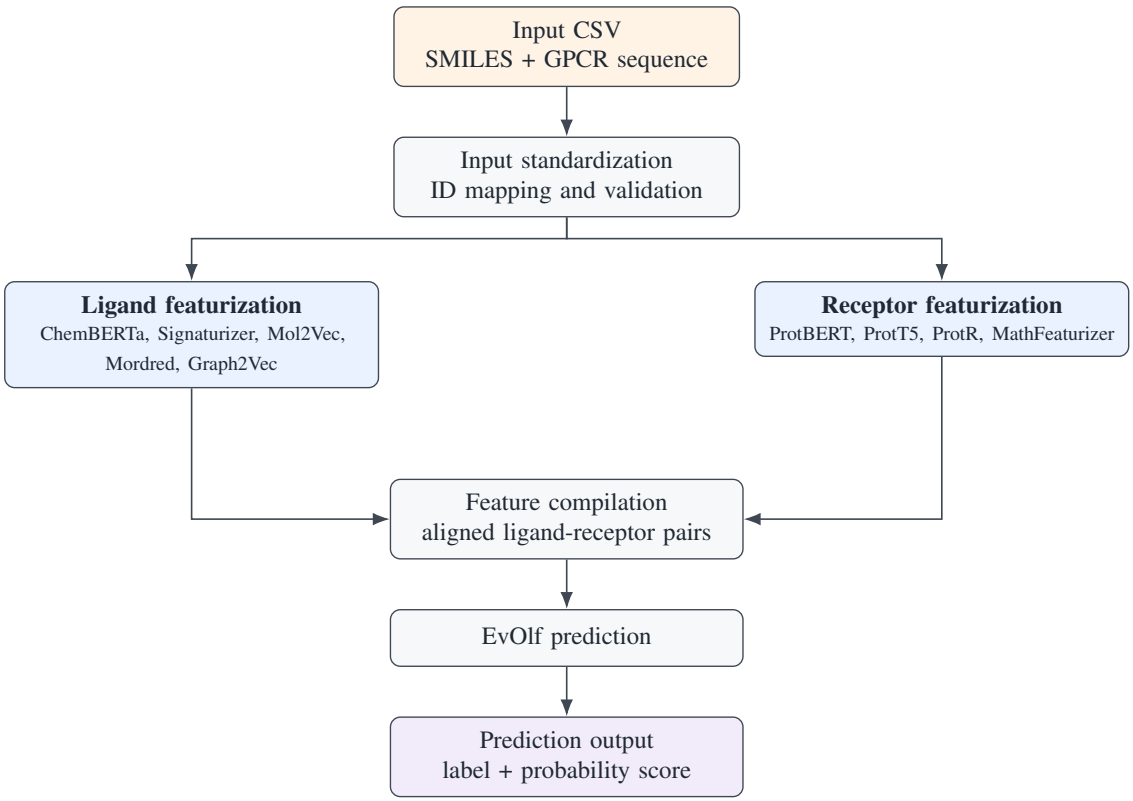

Figure 3.4: EvOlf Nextflow pipeline flow for multimodal GPCR-ligand prediction.

A dedicated Nextflow pipeline was developed to execute locally and on batches. This includes defining the steps involved in input preparation, ligand feature generation, receptor feature generation, feature compilation, and prediction. This enables capturing metadata related to execution, including report, timeline, trace, and DAG information.

3.2 NUCXPLORE PACKAGE AND PIPELINE METHODOLOGY

In the NucXplore project, the goal was to port a Python library for extracting features from segmented nuclei to a Rust library that would be callable from Python using the PyO3 library. This would ensure preservation of the intended feature algorithm while achieving better performance and package integration within reproducible pipelines.

3.2.1 INPUT CONTRACT AND PACKAGE ARCHITECTURE

This package works with RGB image tiles together with MATLAB `.mat` instance masks. The instance mask contains `0` for the background and positive integers for each nucleus. The spatial dimensions of the image and mask should match. In file-based mode, image loading and MATLAB parsing are done by Rust. In memory-based mode, NumPy arrays can be passed from Python.

This package is comprised of Rust modules that handle I/O operations, features calculation,

GPU code, and stains/colors support. PyO3 provides the Rust core as a Python module. This architecture allows for keeping the performance-critical code in Rust but retaining Python compatibility.

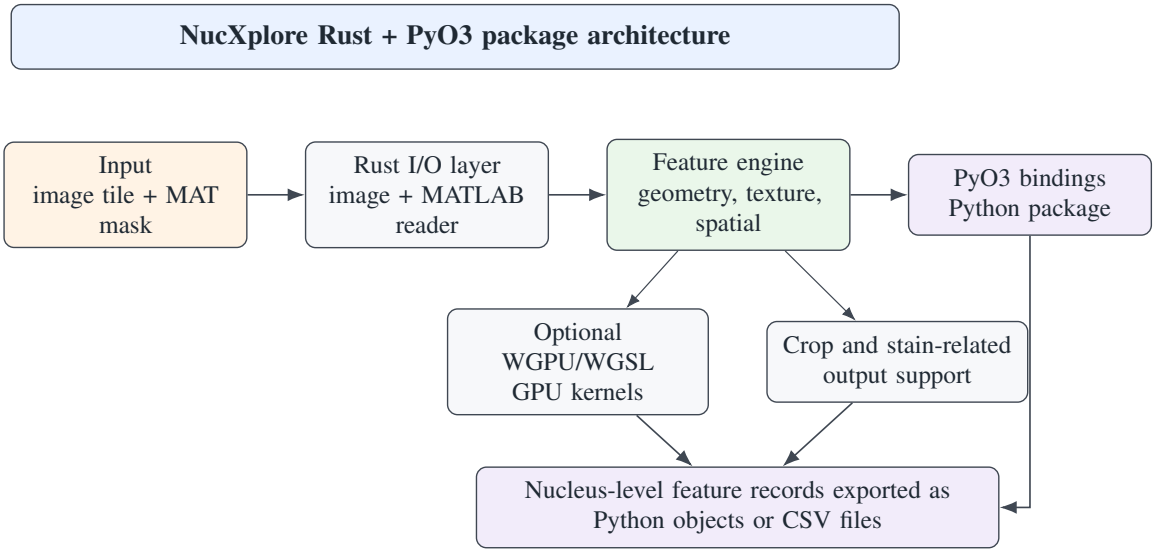

Figure 3.5: Architecture of the NucXplore Rust + PyO3 package for nucleus-level feature extraction.

3.2.2 PUBLIC API MODES

NucXplore allows for various access methods. The recommended access method with minimal dependencies is the function `extract_features_from_files`, which takes the path to the image and the path to the MAT-file as arguments. In-memory access is provided by the function `extract_features`, which takes a NumPy array of images and masks. The batch APIs offer root and samplesheet-based access, and crop export is possible if needed for visual inspection or further analysis at the crop level.

This API design ensures that the user-friendliness is separated from the feature engine implementation in Rust. The user is able to choose the access method suitable for their purposes, while the feature extraction itself is performed in the same way as always.

3.2.3 FEATURE EXTRACTION WORKFLOW

For every image/MAT pair, the workflow detects the nuclei from the instance map and calculates nucleus-level features. These include identity and location features, geometry, shape features, texture features, H&E color features, HOG features, features derived from CCSM, and spatial features. The goal of the Rust implementation was not to re-engineer these features but to implement the expected schema and semantics in a more efficient manner.

Crop export (optional) writes masked nucleus crops to disk. It is useful for inspection purposes

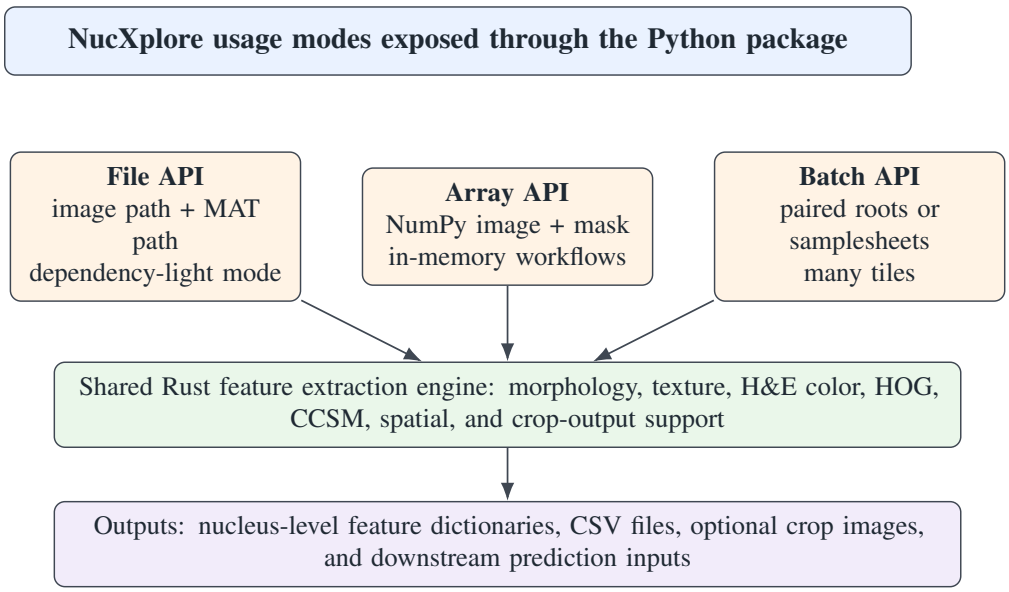

Figure 3.6: Public NucXplore API modes. The file API is used as the preferred dependency-light route in the pipeline, while array and batch APIs support Python integration and high-throughput workflows.

but may slow down the workflow due to many crop images being generated. That is why crop export is implemented as an option of the workflow and is analyzed separately with regard to its runtime impact.

3.2.4 WGPU/WGSL AND PACKAGING

WGPU and WGSL were incorporated to have an alternative GPU-related backend as opposed to a CUDA backend only. The package provides explicit GPU behavior via configuration options, which can either be CPU execution, required GPU execution, or default. This ensures that the package remains functional in the absence of a compatible GPU but still provides the possibility for acceleration where available.

The package was made available as a Python wheel by `maturin`. The packaging method makes it possible for Python developers to easily install and import the package from within Python. File API usage minimizes Python-side image and MAT-file dependencies.

3.2.5 NEXTFLOW PIPELINE DESIGN

The NucXplore process involves four major steps, namely: cropping/filtration, segmentation, feature extraction, and prediction. The process of cropping/filtration involves the partitioning of whole slide images into tiles and discarding any empty or partial tiles. In the process of segmentation, the application of RGCI/HEIP creates instance masks for each tile. The NucXplore package is applied in the process of feature extraction.

Pipeline can execute fully as well as selectively at any stage. The user can choose to execute

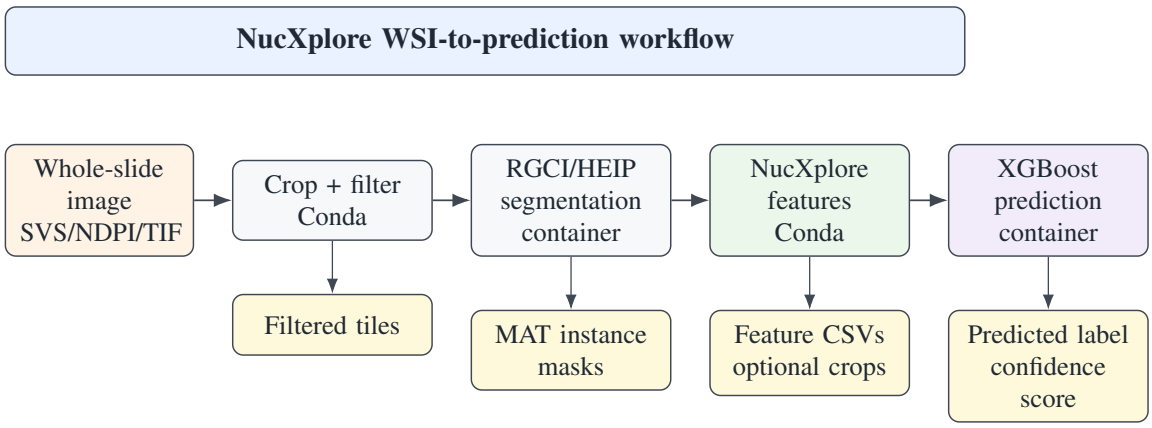

Figure 3.7: NucXplore Nextflow pipeline for whole-slide crop/filtering, segmentation, feature extraction, and cell-type prediction.

the whole pipeline or choose to start the pipeline from any point using any intermediate results like crop tiles, image/MAT file, or feature CSV files. The crop/filtering and feature extraction will be done locally using a Conda environment called `nucxplore-local`, whereas segmentation and predictions will be done using containers.

3.2.6 VALIDATION STRATEGY

Validation was done on both package and pipeline levels. Package level validation involved Rust unit tests, Python API tests, wheel generation, and installation tests. Pipeline level validation was done using Nextflow stub runs and contract tests on stage input/output. Output level validation involved using Docker-built reference CSV files where available.

The validation criteria differentiate between stable outputs and those that are numerically sensitive. Outputs that are not from CCSM are validated using exact matching. Those that are from CCSM are validated using tolerance due to floating point reassociation sensitivity. Same build predictions should have the same prediction label and confidence scores.

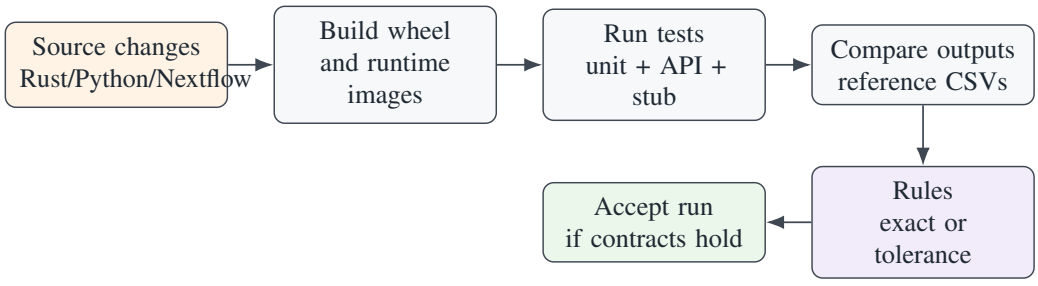

Figure 3.8: Validation workflow used for package and pipeline reliability. Exact comparisons are used where stable, while tolerance is reserved for numerically sensitive CCSM-derived features.

3.2.7 PERFORMANCE EVALUATION SETUP

The entire NucXplore pipeline was tested using `GTEX-1117F-0126.svs` image with a size of 314 MB. The computation was conducted on a Linux server equipped with Intel Core i9-9900KF CPU, 16 logical cores, 96 GB RAM, and NVIDIA RTX A5000 GPU with 24 GB VRAM and CUDA 12.2. This test consisted of timing of crop/segmentation, filtering, features extraction, and classification as individual workflow steps.

This chapter only describes the experiment configuration in the context of methodology. Runtime analysis is provided in Chapter 4.

CHAPTER 4

RESULTS AND ANALYSIS

This chapter presents findings on implementation, pipelines, runtimes, and validations. Because no new models were trained in the dissertation, the findings will not be presented as comparisons in terms of model accuracy. Instead, the focus will be on the success of execution, usability of the output, decreased friction for deployment, and reproducibility.

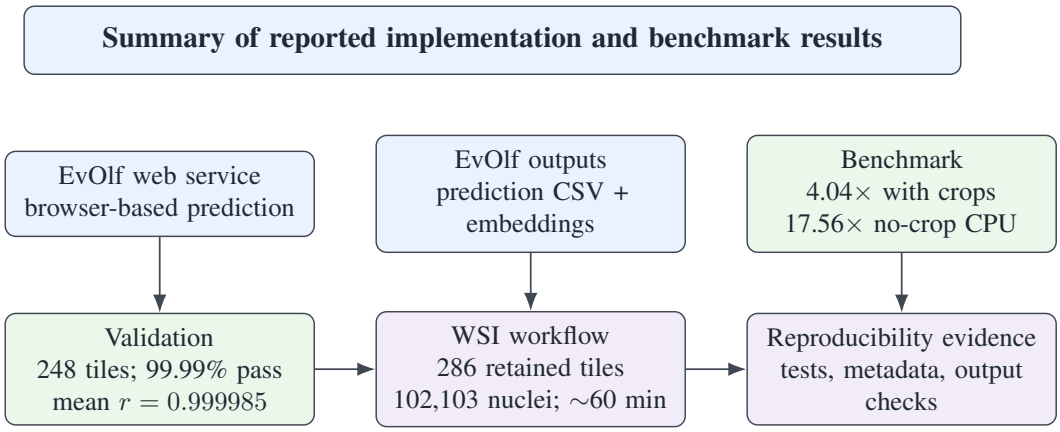

Figure 4.1: High-level result dashboard showing the main implementation, runtime, and validation outcomes reported in Chapter 4.

4.1 EXPERIMENTAL SETUP

EvOlf was assessed as a deployment and workflow system based on representative ligand SMILES and GPCR FASTA inputs. The assessment was to check if the system is capable of receiving valid input, computing required features, running inference, displaying results, producing downloadable output, and executing workflow locally.

NucXplore was assessed via package-level testing and one end-to-end pipeline test for the whole slide image `GTEX-1117F-0126.svs` with a size of 314 MB. This test was performed on a Linux machine with an Intel Core i9-9900KF CPU, 16 logical CPUs, about 96 GB RAM, and NVIDIA RTX A5000 GPU with 24 GB VRAM. Runtime was recorded during cropping/filtering, segmentation, feature extraction, and prediction.

The assessment criteria were thus implementation-focused: successful execution, expected output generation, runtime behavior, and reproducibility. These are consistent with the focus of this dissertation, which is on implementation and optimization.

4.2 EVOLF WEB SERVICE RESULTS

The EvOlf web application turned the prediction pipeline into a web-accessible application. The developed system consists of a frontend based on React/TypeScript, backend using Django, dataset explorer powered by MySQL database, Docker container-based inference services, and NGINX router. The developed system allows one to browse curated GPCR-ligand entries and make single predictions.

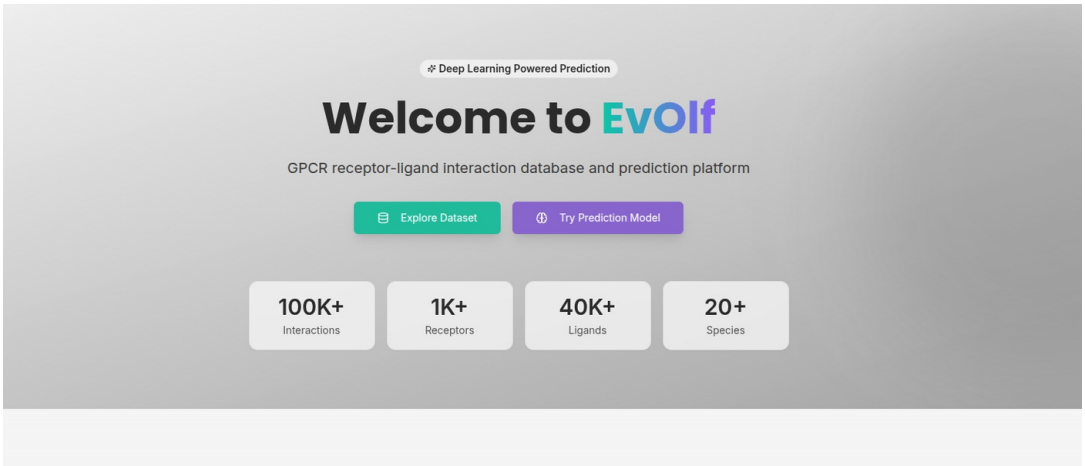

Figure 4.2: EvOlf landing page showing the two main user-facing services: dataset exploration and prediction.

The data explorer offered a useful browsing layer for the dataset associated with the model. In the implemented interface, the explorer showed 110,502 interactions, 1,196 receptors, 24 species, and 41,343 ligands. The search and filtering capabilities helped users examine receptors, ligands, bind/unbind status, mutation type, and database IDs.

A selected record could be opened to view receptor information, ligand structure, mutation status, and external links, as shown in Figure 4.4.

Inputs for the predictor included a plain sequence of the GPCR receptor and its ligand SMILES. The predictor also showed constraints on inputs such as plain amino acid sequences being used in place of FASTA headers, length restrictions for the receptor sequence, SMILES length restrictions, and usage of canonical SMILES if possible.

After submission, the result page reported job status, score, binding status, and the files included in the downloadable archive.

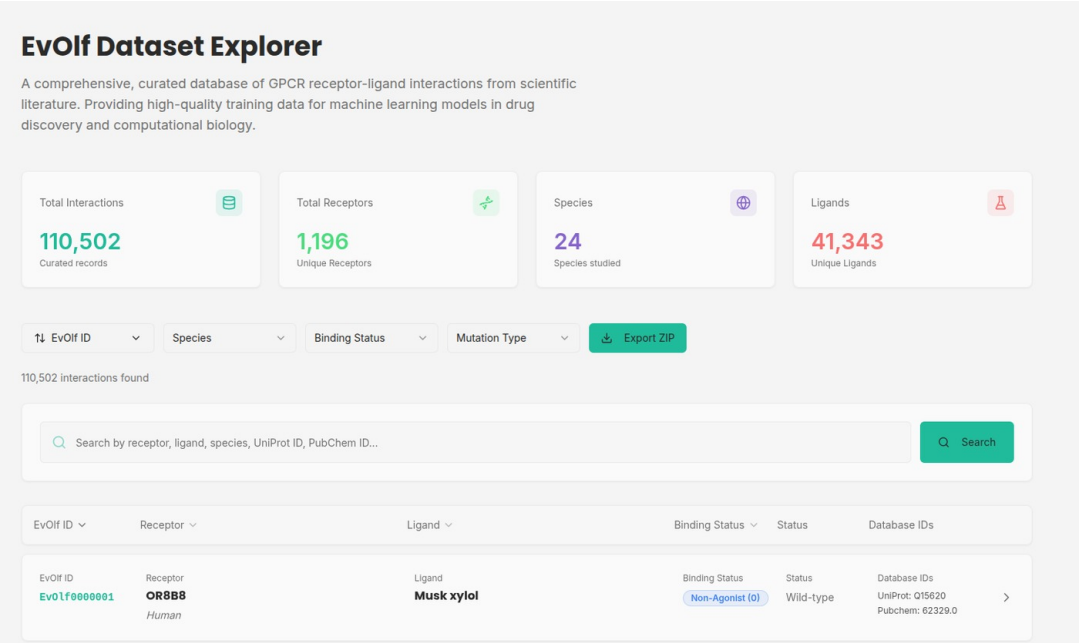

Figure 4.3: EvOlf dataset explorer showing searchable GPCR-ligand interaction records and summary counts.

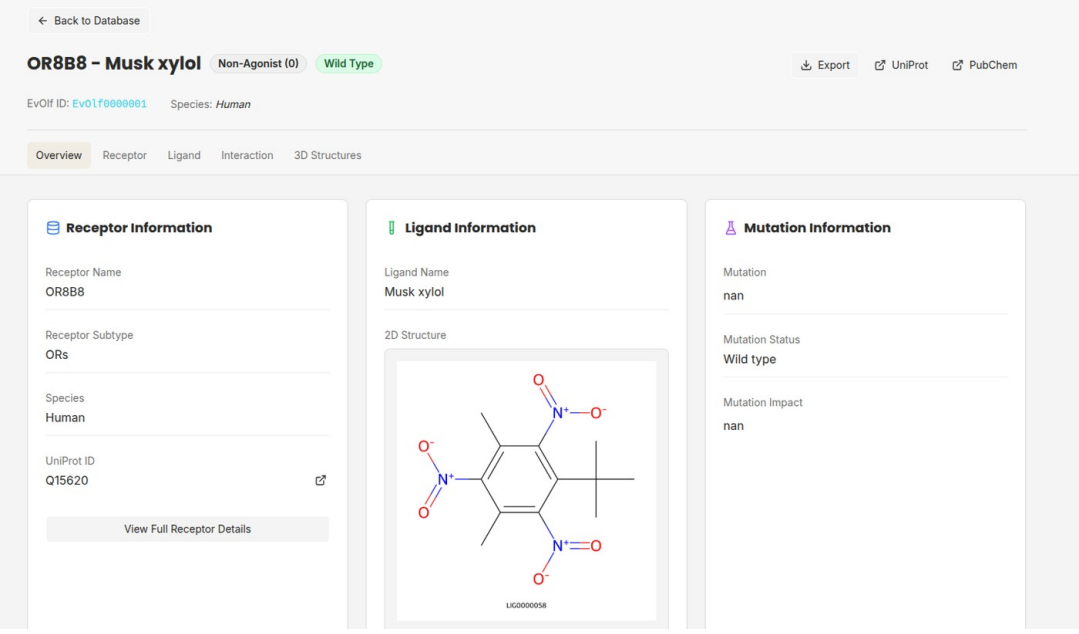

Figure 4.4: EvOlf dataset explorer detail page showing receptor, ligand, mutation, and linked database information for a selected interaction.

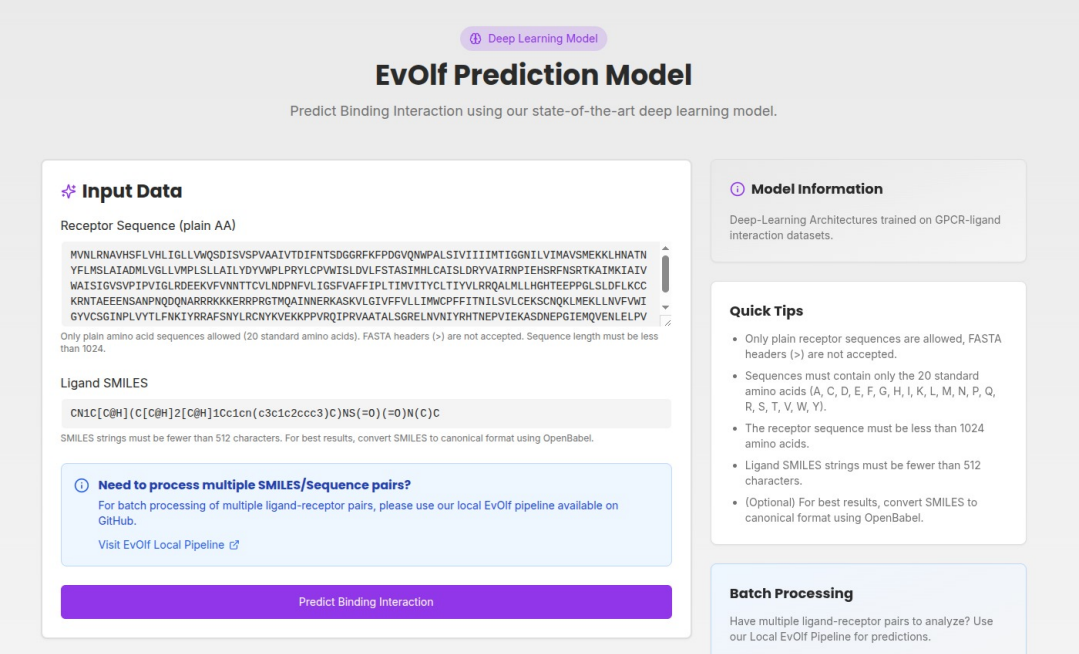

Figure 4.5: EvOlf prediction input page accepting receptor amino acid sequence and ligand SMILES.

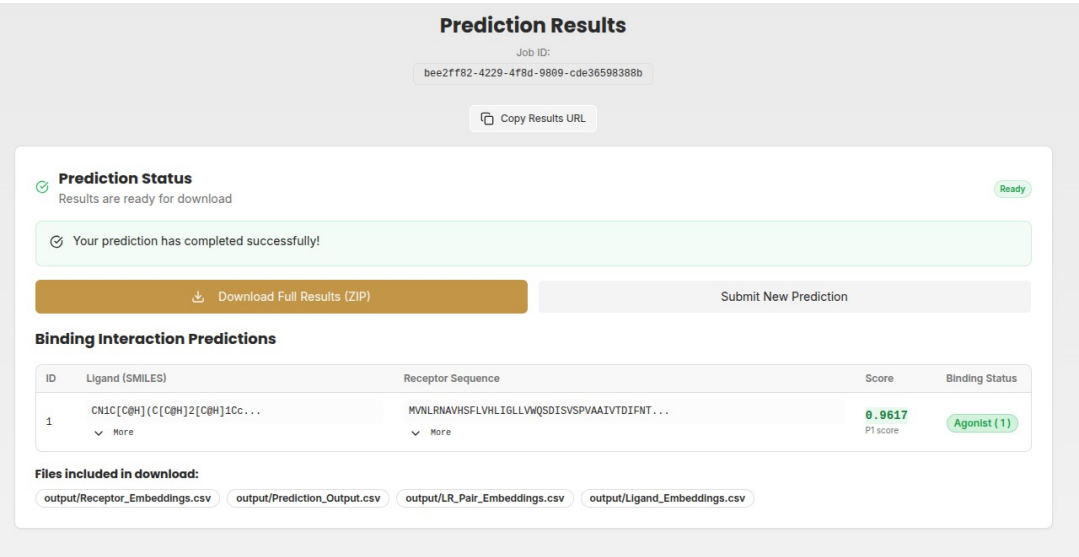

Figure 4.6: EvOlf prediction result page showing completed job status, prediction score, binding status, and downloadable outputs.

In the case of the web-service execution that was carried out, the `Prediction_Output.csv` file had a receptor-ligand pair that gave a prediction value of 1 with a P1 score of 0.9617165327072144. The output on the results page showed the same prediction as an agon

Table 4.1: Observed EvOlf web-service prediction result from the generated `Prediction_Output.csv`

ID	Predicted Label	P1 Score
`tensor(1)`	1	0.9617165327072144

The downloaded result archive contained the four expected CSV outputs used for inspection and downstream analysis: ligand embeddings, receptor embeddings, ligand-receptor pair embeddings, and the final prediction output.

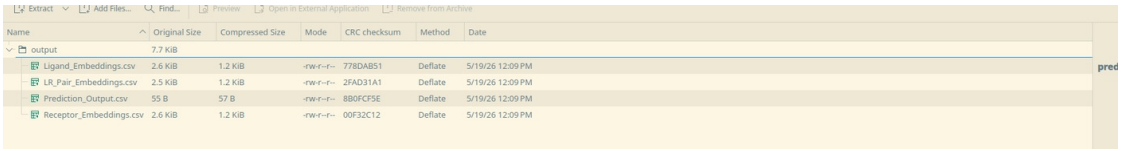

Figure 4.7: Downloaded EvOlf result archive containing ligand embeddings, receptor embeddings, ligand-receptor pair embeddings, and prediction output CSV files.

Table 4.2: Observed files in the downloaded EvOlf web-service result archive

File	Role in the output package
`Prediction_Output.csv`	Final pair-level prediction containing the predicted label and probability score.
`Ligand_Embeddings.csv`	Numerical representation generated for the submitted ligand.
`Receptor_Embeddings.csv`	Numerical representation generated for the submitted GPCR receptor sequence.
`LR_Pair_Embeddings.csv`	Combined ligand-receptor representation used for interaction prediction.

The main result of this component is therefore usability and traceability. The user-facing interface hides the complexity of feature generation and inference while still returning structured files that can be inspected, stored, and reused.

4.3 EVOLF NEXTFLOW PIPELINE RESULTS

EvOlf Nextflow pipeline also enabled a reproducible local workflow for the same predictions. It created a clear workflow definition that involved input preparation, ligand featurization, receptor featurization, feature combination, and finally prediction. It therefore clearly distinguished the web usage case from other batch institutional executions.

In the project README, there is a description of the standard output structure in which each individual input file gets its own result folder with the final predictions CSV file, the input-id map file and other pipeline metadata including execution reports and timings.[21] There is also documentation of the generated embeddings and the final prediction results file format.

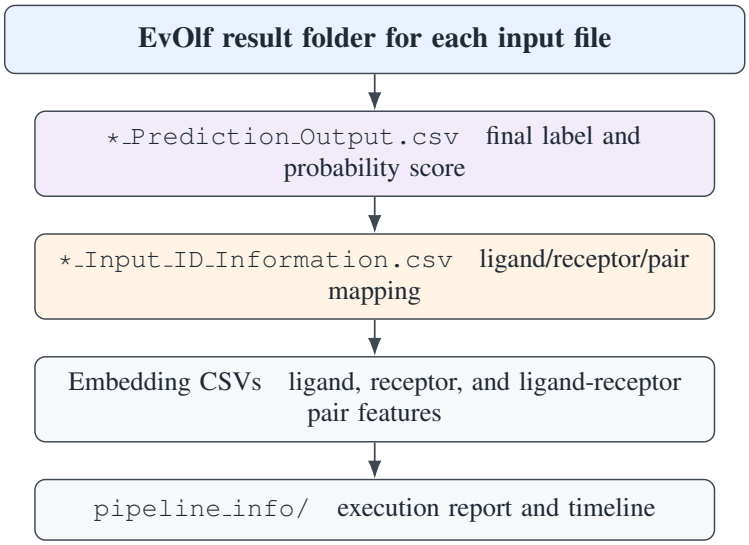

Figure 4.8: Documented EvOlf output layout for local execution and downloadable results.

Table 4.3: EvOlf output files documented in the project README

Output file	Information provided
`Prediction_Output.csv`	Final ligand-receptor pair predictions, including pair ID, predicted label, and probability/confidence score.
`Input_ID_Information.csv`	Mapping file for ligand IDs, receptor IDs, pair IDs, original inputs, and processing status.
`Ligand_Embeddings.csv`	Numerical ligand representations generated during featurization.
`Receptor_Embeddings.csv`	Numerical receptor sequence representations.
`LR_Pair_Embeddings.csv`	Combined ligand-receptor embeddings used for prediction and downstream analysis.
`pipeline_info/`	Nextflow execution reports and timelines for resource usage and provenance.

It is important that the pipeline offers the advantage of non-web execution. The workflow can be run locally or even on powerful computational machines. It thus becomes easier to execute repeated or batch predictions through the workflow.

4.4 NUCXPLORE PACKAGE RESULTS

The NucXplore library was ported from a Python-based featurizer into a Rust library using PyO3 to provide a Python interface. The main result of this conversion is a lightweight wheel

that processes RGB image tiles and MATLAB instance masks in a file-based manner. This means that less effort needs to be put into loading images and MAT files in Python when extracting features while retaining a Python interface for scientific use.

This library supports file-based extraction, NumPy-based extraction, batch extraction, crop export, and GPU operations via WGPU and WGSL. The public API that was benchmarked and validated was `extract_features_from_files`. It loads images and MAT files, extracts nucleus-level features, and saves crops if needed.

Table 4.4: NucXplore package implementation summary

Aspect	Result
Implementation language	Rust
Python interface	PyO3 bindings
Distribution format	Python wheel built with `maturin`
Primary input	RGB image tile and MATLAB instance mask
Primary API	`extract_features_from_files`
GPU-related support	WGPU backend with WGSL shaders
Output	Nucleus-level feature records and CSV files

It is necessary to highlight that the resulting package plays a significant role in improving the featurizer's deployment boundary. While users are still able to import the library using Python, the actual logic for file loading and feature extraction will be implemented in the compiled code in Rust.

4.5 NUCXPLORE VALIDATION AND BENCHMARK RESULTS

A distinct validation and benchmark test was done after the package and pipeline development to assess correctness and runtime behavior using actual image-tile input data. The benchmark made use of 249 PNG image-tiles and 249 corresponding instance-mask images created by MATLAB for the GTEX-1F75B-0126 tile set. Out of these, 248 image-tiles were found to have reference CSV files for their features while one image-tile did not have any nuclei. This latter image-tile was taken into account for runtime computation in the case of the Python implementation but not Rust.

Validation at package level succeeded on all the integration tests with Rust. Pipeline level Python tests have also been successful. Accuracy check at feature level involved the processing of 248 reference tiles and comparison of 10,088,493 numbers. The total success rate was 99.99%, the mean absolute error was 0.00150723, and mean Pearson correlation was 0.999985. This means that the Rust implementation has maintained the numerical behavior of the

Table 4.5: Scope of the real-data NucXplore validation and benchmark run

Item	Value
Dataset	GTEX-1F75B-0126 tile set from `Sample_For_Adnan`
Runtime benchmark inputs	249 PNG image tiles and 249 matching MATLAB instance-mask files
Reference feature CSVs	248 reference CSVs; the zero-nucleus tile was included in runtime accounting but excluded from reference-value validation
Total nuclei processed in benchmark	79,321 nuclei across the tile set
Hardware	Linux workstation with NVIDIA RTX A5000 GPU and driver 535.230.02
Software	NucXplore 0.2.0; Python 3.10.20; Cargo 1.94.0; Nextflow 26.04.0
Benchmark modes	Python with crop export; Rust CPU with crop export; Rust CPU without crop export; Rust GPU/WGPU without crop export

reference feature extraction process up to a very high level, with the rest of the differences due to numerical precision, order of features, metadata columns, and floating point behavior.

Table 4.6: NucXplore validation summary from the final benchmark run

Validation item	Method	Observed result
Environment metadata	Recorded Python, Rust, Nextflow, GPU, Docker, and Conda package versions	NucXplore 0.2.0; Python 3.10.20; Cargo 1.94.0; Nextflow 26.04.0; NVIDIA RTX A5000
Rust integration tests	`cargo test --tests`	117 tests passed; 0 failed
Pipeline Python tests	`pytest tests/ -v`	9 tests passed; 0 failed
Real-data feature comparison	Feature comparison script on 248 reference tiles	10,088,493 compared values; 99.99% pass rate; mean Pearson $r = 0.999985$
Docker reference check	File-list and column comparison with Docker reference outputs	File list matched; column differences were limited to metadata/prediction columns and feature column ordering

For the speed test with real data, there were four different ways of executing the script: the initial Python baseline with crop export; the Rust CPU with crop export; the Rust CPU without crop export; and the Rust GPU/WGPU without crop export. The version of Rust CPU with crop export was considered the fair comparison since both versions exported the nucleus crops to disk. However, the other two Rust implementations without crop export were considered the fast path for workflows where crop output is not needed.

There is a significant decrease in the time required for feature extraction in practice. Compared with the Python baseline, the Rust CPU with crop export path decreased the runtime from

Table 4.7: Real-data NucXplore feature-extraction benchmark on GTEX-1F75B-0126 tiles

Backend	Total (s)	Median/tile (s)	ms/nucleus	Speedup
Python baseline with crops	7217.83	28.6488	91.00	1.00×
Rust CPU with crops	1788.00	7.0398	22.54	4.04×
Rust CPU without crops	410.97	1.5730	5.18	17.56×
Rust GPU without crops	471.62	1.8738	5.95	15.30×

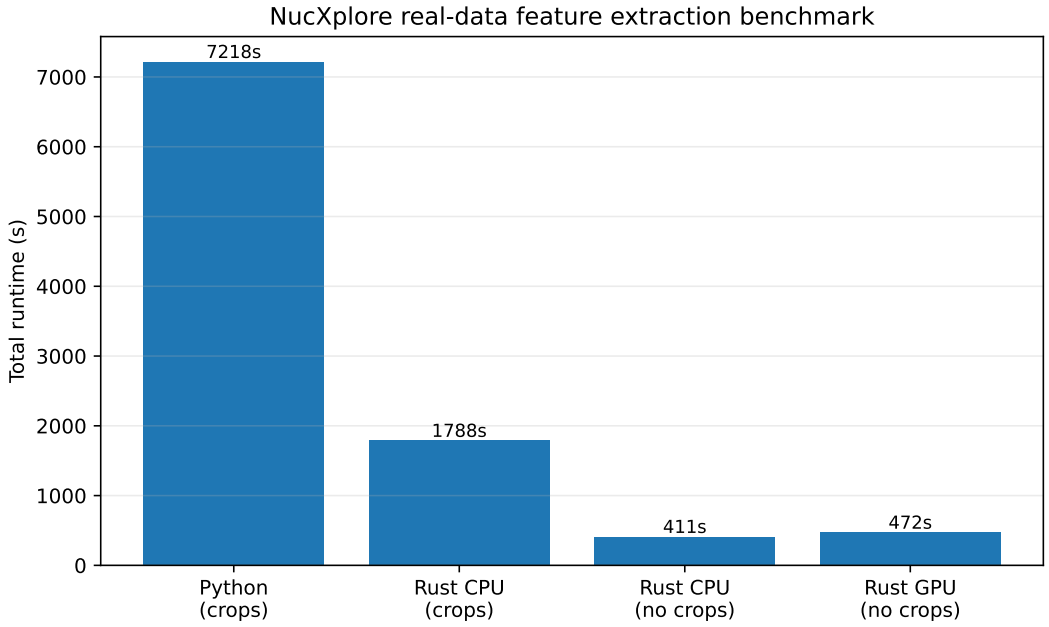

Figure 4.9: Total runtime comparison for the real-data NucXplore feature-extraction benchmark. The Python baseline and Rust crop-enabled run both write nucleus crops, while the no-crop Rust CPU and GPU runs represent fast-path execution when visual crop export is not required.

7217.83 s to 1788.00 s, which gives a total speedup factor of 4.04×. Without using crop export, the runtime of the Rust CPU path dropped even more to 410.97 s, giving a total speedup factor of 17.56×. Meanwhile, the Rust GPU/WGPU without crop export path finished the task in 471.62 s, which was faster than the Python baseline but slower than the Rust CPU without crop export path.

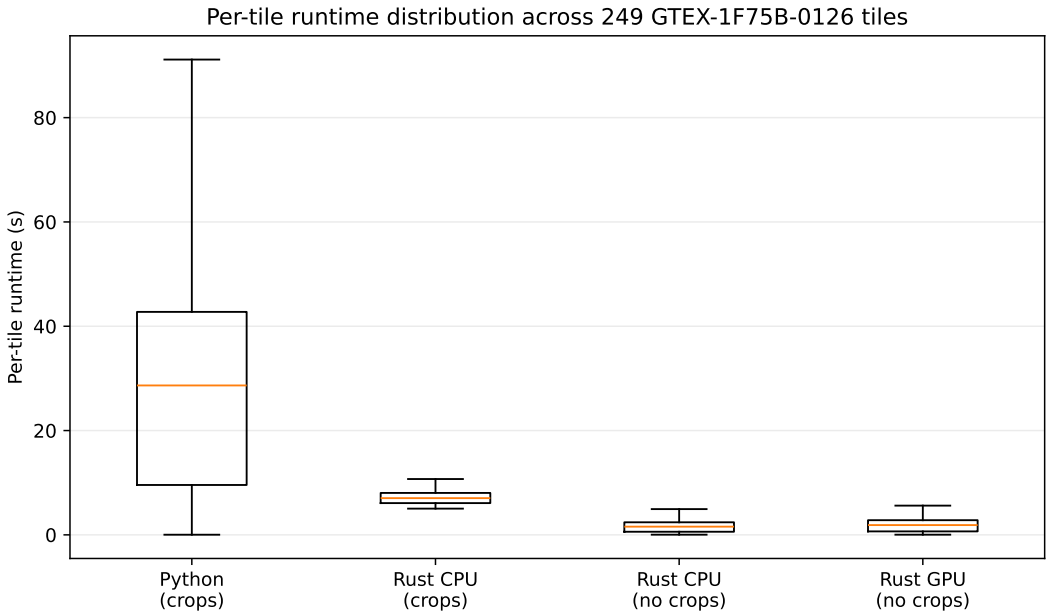

Figure 4.10: Per-tile runtime distribution across the GTEX-1F75B-0126 benchmark tiles. Outliers are hidden in the box plot to keep the central distribution readable.

The per-tile distribution analysis shows that the runtime was indeed decreased by the Rust version for all tiles and not just a few. The runtime was also proportional to the number of nuclei per tile, however, the runtime for the Rust CPU no crop was significantly lower than that of the Python baseline.

The impact of crop export was significant with respect to run time. In the Rust CPU implementation, activating crop export extended run time from 410.97 s to 1788.00 s. The 4.35× increase indicates that disk I/O operations and crop generation have an important influence on actual work flow times. It follows that crop export should be activated for visual auditability but not for feature-only extraction.

4.6 NUCXPLORE PIPELINE RUNTIME RESULTS

The pipeline run included the processing of the WSI using crop/filtering, RGCI/HEIP segmentation, feature extraction, and XGBoost prediction. Crop/filtering produced 1209 tiles and filtered out 286. Feature extraction included 102,103 nuclei using crop export mode.

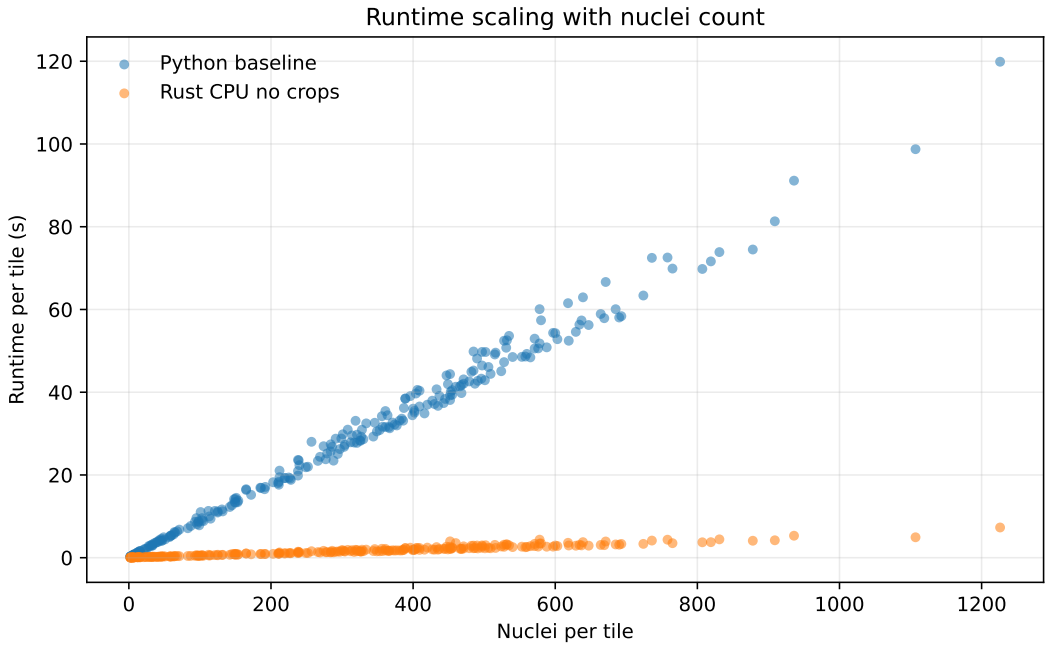

Figure 4.11: Relationship between nuclei count and per-tile runtime for the Python baseline and Rust CPU no-crop execution path. Runtime increases with nuclei count, but the Rust implementation maintains a substantially lower slope across the benchmark set.

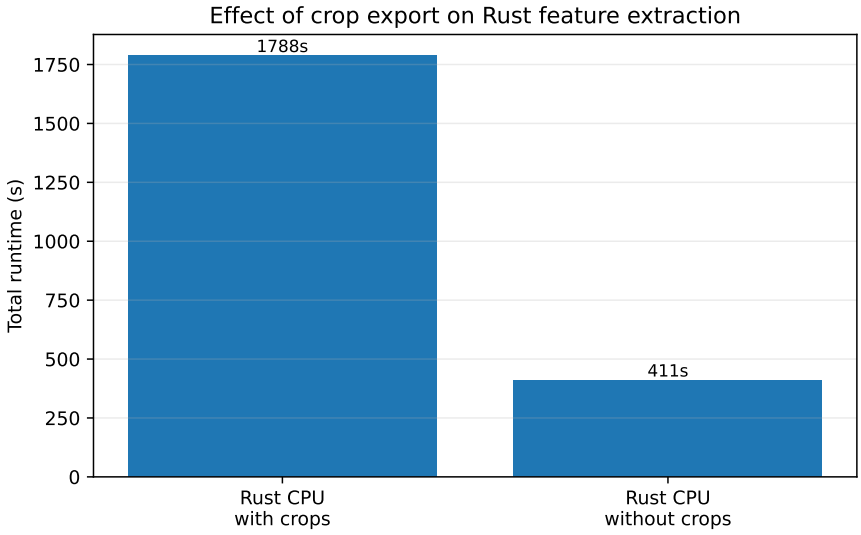

Figure 4.12: Effect of crop export on Rust CPU feature extraction. Enabling crop export increased total runtime from 410.97 s to 1788.00 s, showing that image-output I/O is a major contributor to practical workflow time.

Prediction was done for 285 feature CSVs.

Table 4.8: Full NucXplore pipeline timing for `GTEX-1117F-0126.svs`

Stage	Duration	Main result
Crop/filter	1 min 20 s	1209 tiles generated; 286 retained.
Segmentation	∼28 min	286 retained tiles segmented on GPU.
Feature extraction	30 min 20 s	286 tiles; 102,103 nuclei; crop export enabled.
Prediction	35 s	285 feature CSVs processed.
Total	∼60 min	End-to-end execution including overhead.

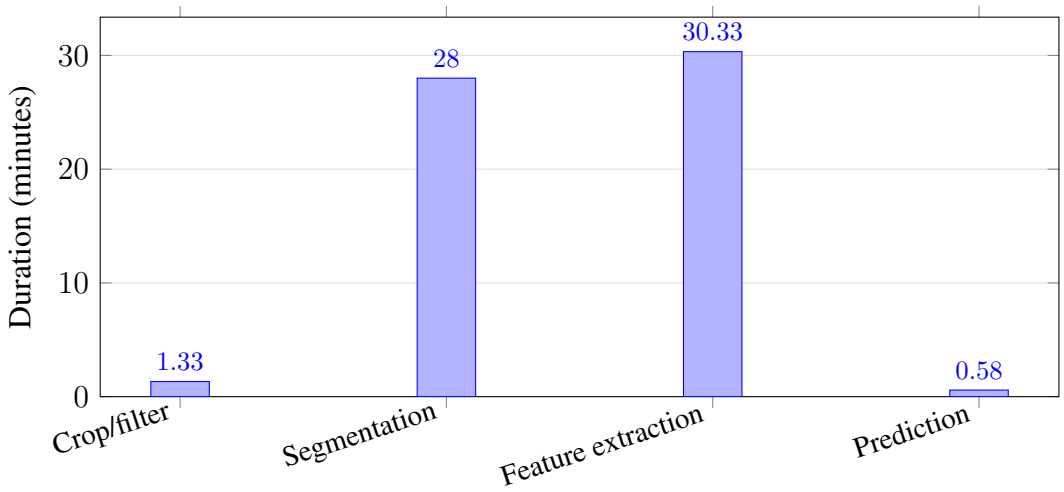

Figure 4.13: Approximate runtime distribution for the NucXplore WSI pipeline run. Segmentation and feature extraction dominate end-to-end runtime, while crop/filtering and prediction are comparatively lightweight.

The performance statistics demonstrate that crop/filtering and prediction are relatively inexpensive operations compared to segmentation and feature extraction. Feature extraction involves both computations and writing CSV files and exporting crop images. Hence, the reported runtime corresponds to real-world pipeline execution rather than only computing features.

The real data benchmark provided above can be used in addition to the end-to-end pipeline runtime for evaluating the performance of the feature extraction. The former only measures feature extraction on pre-generated image/MAT tiles while the latter involves slide tiling, segmentation, feature extraction, prediction, workflow scheduling, and file publication. Both measurements prove the need for optimizing tile-level feature extraction and workflow performance.

4.7 VALIDATION AND REPRODUCIBILITY RESULTS

The validation was carried out at the level of package, pipeline, and output. Package validation involved testing Rust integration for crop export, GLCM behavior, infrastructure, GPU/CPU compatibility, and full pipeline integration. Pipeline validation included testing prediction behavior, stage contracts, explicit boolean parsing, recursive crop detection, empty input behavior, and GPU cleanup behavior. In the last test run, the total number of Rust tests executed was 117 while the number of pipeline python tests executed was 9.

Reference files generated by Docker were also compared against Sample_for_Adnan feature outputs. There were no differences detected between the two file lists; this means that there were 248 feature files in both locations. Column comparison revealed differences arising from metadata columns, prediction columns, and order of feature columns. This does not mean that numeric feature comparison was invalid. However, it reveals that schema comparison must take into account normalization of metadata and predictions columns first.

Validation results lend weight to the reproducibility claim made by the dissertation. Exact reproduction is ideal when possible; however, there are instances in scientific processes where numeric calculations take place, and as such, some level of floating-point variance is possible. It is imperative that validation efforts identify what needs to be exact, what needs tolerance, and what are metadata columns.

Validation layer	What is checked	Expected result
Package tests	Rust tests and Python API checks	APIs compile and feature records are produced
Pipeline tests	Nextflow stubs and prediction tests	Stage contracts and output paths remain valid
Reference outputs	Feature and prediction CSV comparison	Exact match where stable; tolerance for sensitive CCSM features

Figure 4.14: Validation matrix used to interpret package-level, pipeline-level, and output-level checks.

4.8 DISCUSSION OF FINDINGS

These results reinforce the core thesis of the dissertation: the usefulness of an AI system depends on the engineering around the model. The accessibility of EvOlf increased via the web UI and reproducibility via Docker and Nextflow. The accessibility of NucXplore was increased via its Rust/PyO3 implementation.

NucXplore benchmark also proves the importance of optimization on the proper level. The Rust implementation showed 4.04× total-speed increase over the Python baseline in case of crop export. In case when crop export was turned off, the Rust CPU path showed 17.56× total-speed increase. Yet, the total WSI pipeline suffers from such operations as segmentation, disk I/O, crop export, CSV writing, and workflow scheduling. That means that future optimizations should target the whole execution path and not just the innermost feature computation step.

Another lesson in practice from the GPU findings is that while the WGPU was working well on the RTX A5000, the Rust CPU no-crop method was faster than the GPU no-crop method in terms of the full tile processing task. The implication of this observation is that GPU acceleration should be employed only where the overhead costs are less than the computational benefits.

In both case studies, the greatest takeaway is not a novel biological model but rather a more usable computational system. In summary, this thesis shows how easy-to-use interfaces, reproducibility, validation, and optimization can make AI biomedical systems more usable for scientific research.

CHAPTER 5

DISCUSSION

5.1 OVERVIEW OF FINDINGS

Conclusions from the presented dissertation work suggest that practical relevance of artificial intelligence approaches to drug discovery is related not only to the training of models, but also to the software infrastructure around it. The two case studies both support the same thesis – the value of a computational model increases if it is available, reproducible, inspectable and executable within a formalized workflow. EvOlf supports the thesis with its deployment and workflow formalization, while NucXplore does so with its package optimization, workflow integration, validation and performance benchmarking.

As for the main result of EvOlf work, it is worth mentioning that the computational workflow turned into an easily accessible prediction system thanks to the web interface where users could input ligands in SMILES format and receptors in amino acid sequences without any need to deal with scripts and models themselves. The output result page, downloadable archive and dataset explorer made the tool even easier to use and re-use.

For NucXplore, the primary achievement involved the implementation of the workflow for feature extraction in Python to Rust + PyO3 and subsequent incorporation into the Nextflow pipeline. As per the final real data benchmarking results, there was a speedup of 4.04$\times$ with crop export and 17.56$\times$ without crop export using Rust CPU against the crop enabled Python baseline. It should be noted that even in the case of full pipeline results, the overall runtime is dictated by segmentation, crop export, disk I/O, and workflow scheduling.

5.2 ENGINEERING CONTRIBUTION BEYOND MODEL DEVELOPMENT

An important point of this work is the difference between developing an AI model versus implementing it into a production system. The dissertation does not attempt to retrain the EvOlf predictive model, redesign the NucXplore biological feature set definition, or develop a new segmentation architecture. The contribution of this work is to make the existing AI

models more easily usable, distributable, and reproducible.

This difference is crucial, because many AI models remain at the level of research prototypes, despite being scientifically sound, due to difficulties in deploying them into production environment. Users may encounter dependency problems, lack of information about input and output parameters, non-portable scripts, etc. This work solves these issues by using a web service, Docker, Nextflow workflows, Rust, and validation process.

The examples above thus demonstrate that software engineering is not a secondary issue in applied AI for drug discovery. In fact, it plays an important role in determining whether or not a model can be employed by scientists who did not participate in its creation. An application which cannot be easily installed, run, or analyzed is of little use in practice. On the other hand, a good engineering solution can make an existing model more useful.

5.3 DISCUSSION OF THE EVOLF IMPLEMENTATION

The use of EvOlf clearly shows the significance of having accessible interfaces in AI tools used in drug discovery. The online service allows prediction to be made by using the browser interface for each individual ligand–receptor pair and displays the output in a manner that the user can evaluate instantly. In addition, the downloaded archive includes intermediate embeddings and predictions.

The data explorer provides a second dimension of utility through the possibility of browsing pre-curated interaction records between GPCR and ligands. This can be important since a lot of times users of a predictive model would want to know what kind of data the predictive system relies upon. In this regard, the web interface is not just exposing an inference endpoint; it makes the surrounding data easier to interpret.

Whereas the web service caters to one specific user scenario, the Nextflow pipeline solves a separate set of problems. For instance, while the web service is fit for interactive prediction, the pipeline may serve better for local execution, batch prediction, and institutional settings. The reason for this dual mode design is that not all users would want to access the system via the same interface.

5.4 DISCUSSION OF THE NUCXPLORE IMPLEMENTATION

The NucXplore analysis highlights the importance of considering package-level optimization along with pipeline-level execution. The Rust + PyO3 package helped in enhancing the distribution and runtime performance of the package without losing its Python compatibility.

It is crucial to note that most biomedical researchers continue to operate through Python environments despite the possibility of optimizing performance-critical code using low-level language.

The results of the benchmark test indicate that the proposed method performs better than the initial Python-based feature extraction process. For instance, the Rust CPU implementation with crop export enabled resulted in a decrease in total runtime from 7217.83 s to 1788.00 s. It was more appropriate since the two implementations involved writing of nucleus crops to file system. However, the absence of crop export in the Rust CPU implementation further reduced the total runtime to 410.97 s. The method is faster especially when there is no need for visual crop image production.

The GPU timings must be considered carefully as well. The GPU was supported via WGPU on the RTX A5000, with the no crop GPU path completing in 471.62 seconds, a 15.30× improvement over the Python baseline. It was still slower than the Rust CPU no crop path for the full per tile workload, but this does not render the GPU approach invalid. Instead, it demonstrates that the usefulness of the GPU is contingent on the size of the workload, batching, data transfer costs, and the type of operations being accelerated.

5.5 PRACTICAL IMPLICATIONS

Implication of the EvOlf paper is that an advanced multimodal predictive modeling framework can become accessible using the approach based on web deployment and reproducible workflow execution. Implication of the NucXplore paper is that computational biology workflows can be provided with the benefits of low-level package development despite Python interface exposure to users.

This paper also emphasizes the distinction between optimization at the level of tiles and optimization at the level of workflows. The feature extraction was made faster on the real data benchmark, but the whole workflow may still be constrained by the segmentation step, disk I/O, cropping, or scheduling latency. As such, future optimizations will need to consider the whole workflow, not just individual functions.

5.6 LIMITATIONS

The primary weakness of this dissertation lies in the fact that the project does not involve any model development but rather implementation and optimization. The EvOlf model structure and training procedure have remained unchanged. Likewise, the biological feature definitions,

Table 5.1: Practical implications of the two case studies

Area	Implication
EvOlf web service	Reduces the need for users to manually run prediction scripts for single ligand–receptor pairs.
EvOlf dataset explorer	Allows users to inspect curated GPCR–ligand records used around the prediction platform.
EvOlf Nextflow pipeline	Supports reproducible local and batch execution outside the web server.
NucXplore Rust package	Reduces Python dependency complexity while preserving Python access through PyO3.
NucXplore pipeline	Connects crop/filtering, segmentation, feature extraction, and prediction in a structured workflow.
Validation strategy	Separates exact reproducibility from tolerance-based numerical comparison where floating-point sensitivity is expected.

segmentation model, and subsequent XGBoost predictor in NucXplore were all defined independently of the current research.

The provided timing data is also highly dependent on hardware and input parameters. Runtime of whole-slide image analysis may depend on slide dimensions, tissue presence, number of tiles, GPU availability, disk performance, cropping and exporting options, as well as system workload. The NucXplore benchmark for tile-based feature extraction thus needs to be considered separately from overall runtime.

One problem with this thesis is that the research is concerned with the practicality of implementation and replication, but not with biological validation of results. The goal of these tools is to facilitate the execution and replication of existing software, but biological validation would be necessary for new results.

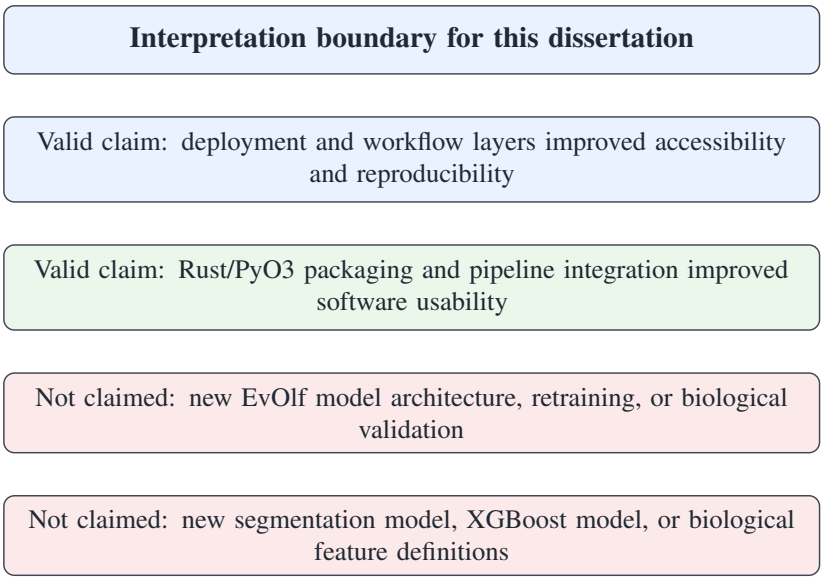

Figure 5.1: Boundary used to interpret the dissertation results. The work is evaluated as an implementation and optimization contribution rather than as new biological model development.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 CONCLUSION

In this dissertation, practical software layers have been built around existing AI-based drug discovery and biomedical image analysis frameworks. For EvOlf, a web frontend, dataset browser, Docker-based inference server, and Nextflow workflow were designed to enhance accessibility and reproducibility of GPCR-ligand prediction. For NucXplore, a Python-based segmented nucleus featurizer was ported to a Rust + PyO3 wheel with WGPU/WGSL support and included in a Nextflow workflow for whole-slide imaging.

As shown by the results, engineer-driven optimization may lead to improved usability, portability, reproducibility, and run-time efficiency of bioinformatics pipelines without modifying the biological model itself. As the first use case example illustrates, web deployment and workflow formalization may enhance the accessibility of a prediction framework. As the second use case example illustrates, a performance-sensitive featurizer may be repackaged and accelerated in an efficient way while still being usable from Python. In the end, the benchmark shows 4.04$\times$ speedup for Rust CPU with crop export and 17.56$\times$ speedup for Rust CPU without crop export over the Python baseline.

However, the validation and benchmark results illustrate that reproducibility is not a monolithic concept but requires the presence of well-defined inputs, controlled environment, well-structured workflow steps, traceable outputs, and adequate validation criteria. While exact comparison is justified for stable outputs, tolerance-based comparison is more realistic for features characterized by numerical sensitivity.

In conclusion, the presented dissertation contributes to the notion that AI-based approaches for drug discovery and biomedical analysis must be considered as computational instruments rather than predictive models alone. An instrument that is hard to deploy, compute, reproduce, and inspect is practically useless. In turn, proper deployment and workflow layer design can enhance the utility of existing models for the greater benefit of the biomedical community.

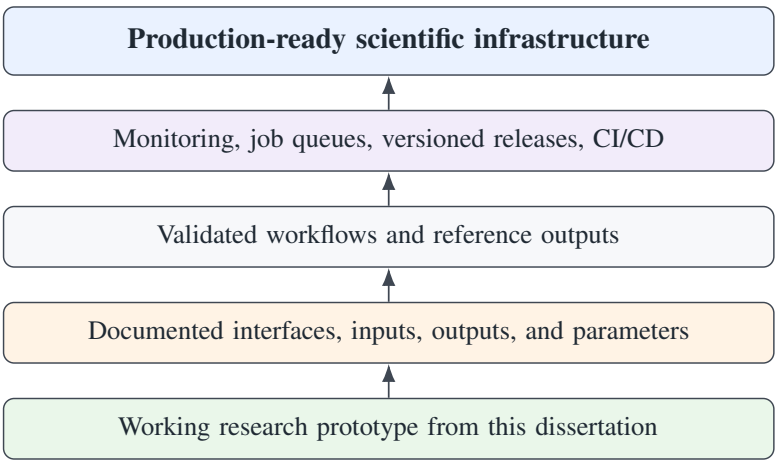

Figure 6.1: Path from the dissertation implementation to production-ready scientific infrastructure. The current work establishes the research-prototype and documented-workflow layers.

6.2 SUMMARY OF CONTRIBUTIONS

The work done in this dissertation can be described via two inter-related implementation strands. First strand involved EvOlf, which resulted in web service application for users, data set explorer, downloadable prediction results, service packaging in containers and Nextflow workflow running locally. Second strand involved NucXplore, which resulted in Rust + PyO3 package, feature extraction from files API, support for crop exports, WGPU/WGSL implementation and Nextflow pipeline integration.

Combining these, four key practical properties of an AI system were addressed: accessibility, reproducibility, performance and traceability. Accessibility was achieved via web interfaces and Python binding. Reproducibility was achieved via Docker and Nextflow, as well as versioned environment and validation output. Performance was achieved via Rust implementation and benchmarking on real data, involving crop-enabled, no-crop and GPU execution paths. Traceability was achieved via output archiving, CSV schema, pipeline report and validation log.

6.3 FUTURE SCOPE

Future work could take both approaches to become more mature scientific infrastructure. Potential improvements to EvOlf might involve public availability of the system, batch file uploads, job queuing system for time-consuming predictions, user authentication, monitoring, and result visualization. A job queuing backend would prove helpful particularly for computationally intensive predictions as users would be able to submit a job and get their results after its completion without an open browser window.

Potential future work on NucXplore would involve benchmarking using more large-scale WSI data, minimization of I/O overhead in the process of cropping, more comprehensive GPU acceleration support, and even distribution/multi-GPU computing. As segmentation and feature extraction account for the majority of computational effort, optimizations at the pipeline level would be more effective than optimization of the feature extraction functions themselves.

Biological validation for the optimized workflows would be an additional means of improving their scientific value with input from domain partners. Such biological validation could involve comparing results for different tissue samples, under different staining conditions, for different segmentation parameters, and during different prediction applications.

6.4 CLOSING REMARKS

The following phase of development should consist of three major steps. Firstly, the systems have to be improved in terms of job handling, documenting, versioning, and monitoring. Secondly, they have to be benchmarked on other data sets using different hardware configurations. Finally, the results have to be discussed with domain experts in order to relate software improvements to biological interpretations.

Thus, the main contribution of this dissertation lies in engineering, rather than development of new prediction models. The proposed approaches are meant to make existing AI solutions for drug discovery more accessible and efficient. This is a necessary step on the way towards the creation of reliable scientific infrastructure based on AI.

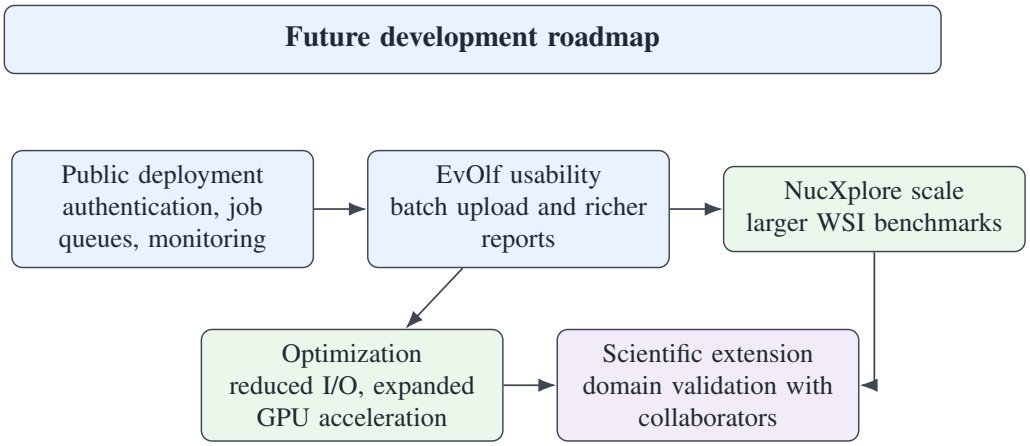

Figure 6.2: Practical roadmap for extending the work beyond the dissertation. The next steps move from research prototypes toward production readiness and broader validation.

REFERENCES

1. Wratten, L., Wilm, A. & Goecks, J. Reproducible, scalable, and shareable analysis pipelines with bioinformatics workflow managers. *Nature Methods* **18,** 1161–1168 (2021).

2. Moreau, D., Wiebels, K. & Boettiger, C. Containers for computational reproducibility. *Nature Reviews Methods Primers* (2023).

3. Mittal, A., Gupta, M., Mohanty, S. K., *et al.* Evolutionary-guided advanced deep-learning architecture powers mammalian GPCRome agonist predictions. *Cell Reports* **45,** 117003 (2026).

4. Chandrasekaran, S. N. *et al.* Image-based profiling for drug discovery: due for a machine-learning upgrade? *Nature Reviews Drug Discovery* (2021).

5. Caicedo, J. C., Cooper, S., Heigwer, F., *et al.* Data-analysis strategies for image-based cell profiling. *Nature Methods* **14,** 849–863 (2017).

6. Wilson, G., Aruliah, D. A., Brown, C. T., *et al.* Best practices for scientific computing. *PLoS Biology* **12,** e1001745 (2014).

7. Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., *et al.* The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data* **3,** 160018 (2016).

8. Lewis, J. *et al.* Replication, reproducibility, and other issues in computational biology. *BMC Systems Biology* (2016).

9. Fell, J. *et al.* Reproducibility of deep learning algorithms for digital pathology. *PLOS Digital Health* (2023).

10. Rasool, A., Rahman, J. U. & Uwitije, R. Enhancing molecular property prediction with quantized GNN models. *Journal of Cheminformatics* **17,** 81 (2025).

11. Lu, X. *et al.* Multimodal fused deep learning for drug property prediction: Integrating chemical language and molecular graph. *Computational and Structural Biotechnology Journal* **23,** 1666–1679 (2024).

12. Besiroglu, T. *et al. The compute divide in machine learning* MIT FutureTech. 2024.

13. Boettiger, C. An introduction to Docker for reproducible research. *ACM SIGOPS Operating Systems Review* **49,** 71–79 (2015).

14. Di Tommaso, P. *et al.* Nextflow enables reproducible computational workflows. *Nature Biotechnology* **35,** 316–319 (2017).

15. Goecks, J., Nekrutenko, A. & Taylor, J. Galaxy: a comprehensive approach for supporting accessible, reproducible, and transparent computational research in the life sciences. *Genome Biology* **11,** R86 (2010).

16. Afgan, E., Baker, D., Batut, B., *et al.* The Galaxy platform for accessible, reproducible and collaborative biomedical analyses: 2018 update. *Nucleic Acids Research* **46,** W537–W544 (2018).

17. Bankhead, P., Loughrey, M. B., Fernandez, J. A., *et al.* QuPath: Open source software for digital pathology image analysis. *Scientific Reports* **7,** 16878 (2017).

18. HistomicsTK Developers. HistomicsTK: A toolkit for computational pathology. *SoftwareX* (2025).

19. McQuin, C., Goodman, A., Chernyshev, V., *et al.* CellProfiler 3.0: Next-generation image processing for biology. *PLoS Biology* **16,** e2005970 (2018).

20. Carpenter, A. E., Jones, T. R., Lamprecht, M. R., *et al.* CellProfiler: image analysis software for identifying and quantifying cell phenotypes. *Genome Biology* **7,** R100 (2006).

21. Ahuja Lab. *EvOlf: Evolutionary-Guided Advanced Deep Learning Architecture Powers Mammalian GPCRome Agonist Predictions* `https://github.com/the-ahuja-lab/EvOlf` (2026).

APPENDIX / ORIGINALITY REPORT

Attach the Turnitin similarity report and any required university originality documentation here.

SUPPLEMENTARY EVOLF OUTPUT ARTIFACTS

The supplementary files included with this thesis bundle contain the web-service result artifacts used in Chapter 4:

- `data/evolf/Prediction_Output.csv`: final prediction output from the tested EvOlf web-service run.
- `data/evolf/Ligand_Embeddings.csv`: ligand embedding output generated for the submitted ligand.
- `screenshots/`: raw EvOlf interface screenshots used to prepare the figures in Chapter 4.

SUPPLEMENTARY NUCXPLORE VALIDATION ARTIFACTS

The NucXplore validation files used to prepare the Chapter 4 benchmark tables and figures are included under `validation_artifacts/`. The most important files are:

- `nucxplore_validation.zip`: complete uploaded validation archive.
- `nucxplore_benchmark_summary.json`: aggregate benchmark metrics used in the speed benchmark table.
- `nucxplore_benchmark_per_tile.csv`: per-tile timing data used for runtime distribution and scaling plots.
- `nucxplore_accuracy_validation.txt`: terminal output from the feature-value comparison against Python reference CSVs.

- `nucxplore_rust_tests.txt` and `nucxplore_py_tests.txt`: package and pipeline test logs.
- `nucxplore_environment_metadata.txt`: software, hardware, and environment metadata for reproducibility.